Fundamentals of Deep Learning and Programming

Lecture 2: Neural Network Basics

Fall Semester 2025

Adjunct Professor: Donghoon Kang

Learn from Data

Future Data

Model

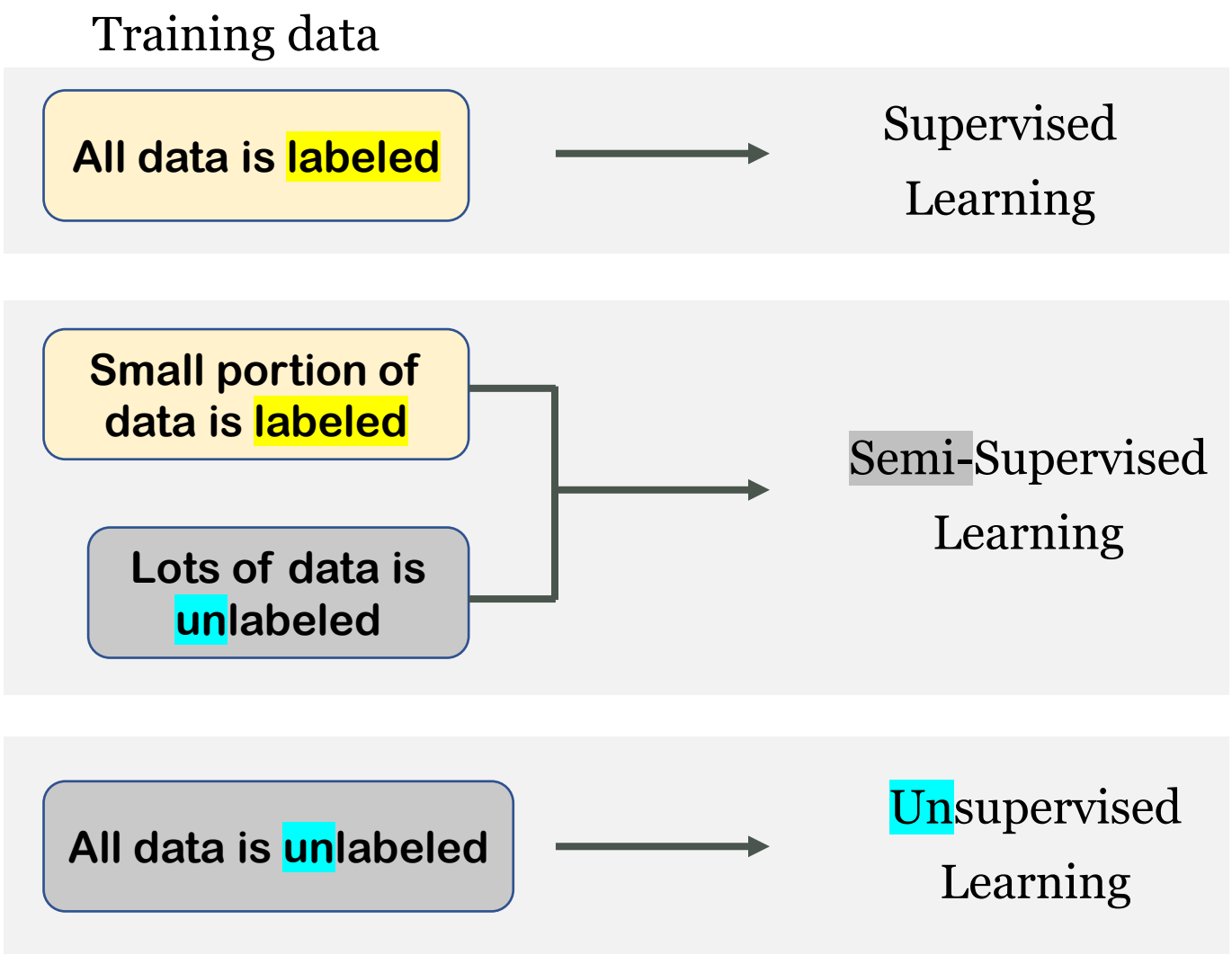
Predict

"car"

Train

Past Data
(perhaps with **labels**)

Supervised, Semi-Supervised, and Unsupervisd Learning



Learn a function f to **map** data x to label y:
$y = f(x)$

Learn some underlying **hidden structure of data**

Problem: Supervised Learning is NOT how we humans learn

Baby don’t get supervision for everything they see !

Self-Supervised Learning

- Self-supervised learning is a machine learning paradigm where the model learns from unlabeled data by creating its own supervised signals.

  - In self-supervised learning, the model generates “labels” automatically from the raw data and then trains itself as if it were a supervised task.

  - Self-supervised learning is generally considered unsupervised learning in terms of **data** (no manual labels), but supervised learning in terms of **method** (training with input–output prediction tasks).

Rapid Generalization and Efficient Learning like Human

Traditionally, a deep learning model needs thousands, or even millions, of examples to learn a new task. But what if we don't have enough data?

- This is where Zero-shot, One-shot, and Few-shot learning come in. These concepts are inspired by how humans learn—we can often recognize a new concept with very few, or sometimes zero, examples.

Zero-shot, One-shot, and Few-shot leverage **transfer learning** and **meta-learning (=learning to learn)** to enable models to generalize with minimal data, a major step towards more intelligent AI.

Zero-Shot Learning (ZSL)

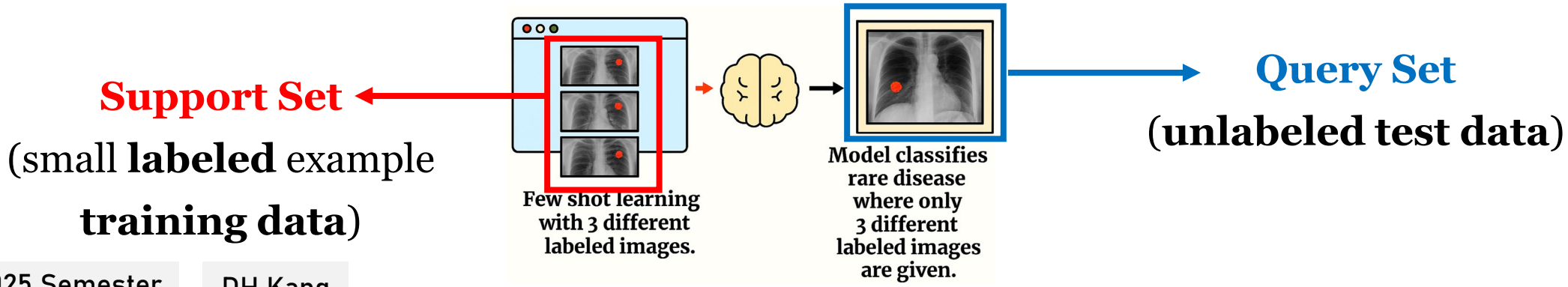
The model should classify an object from a class it has never seen before during training.

One-Shot Learning (OSL)

The model is given only one example of a new class and must classify a new instance of that class.

Few-Shot Learning (FSL)

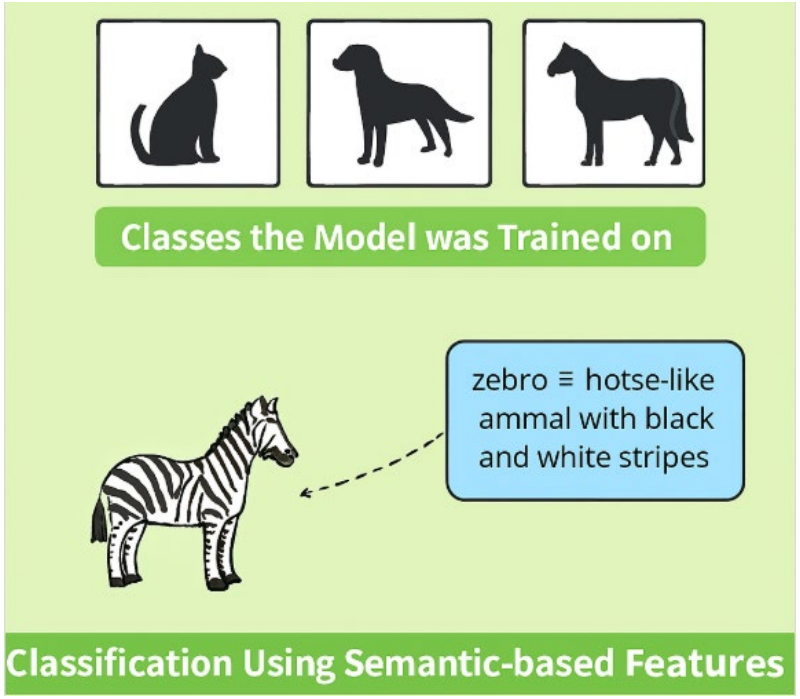
The model is given a small number (e.g., 3-5) of examples per new class and must classify a new instance of that class.



Zero-Shot Learning

Zero-shot learning means the model can recognize a new class **without ever seeing any training example of it**. Instead, it uses semantic descriptions or prior knowledge.

Example: The model has never seen a zebra photo. But if it knows that a zebra is “a horse-like animal with black and white stripes”, it can use this description to classify zebras correctly, even without any zebra training image.



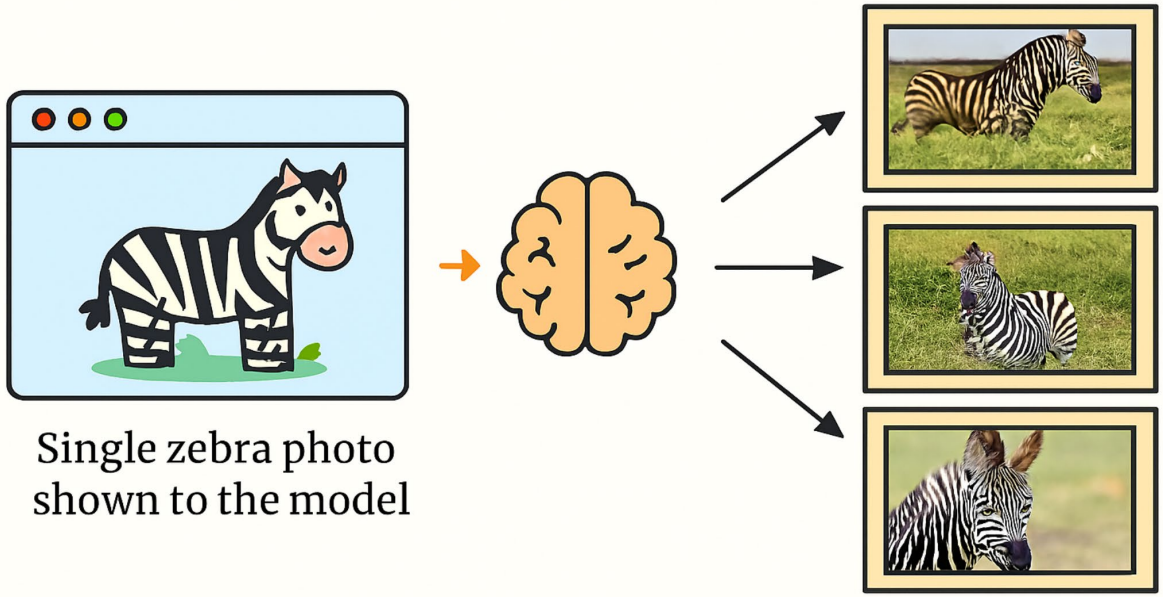
Another **Examples**:

- Machine Translation: Translating from English to German without having seen parallel training data for that specific language pair.
- Prompting LLMs: Asking ChatGPT to solve a task without examples in the prompt.

One-Shot Learning

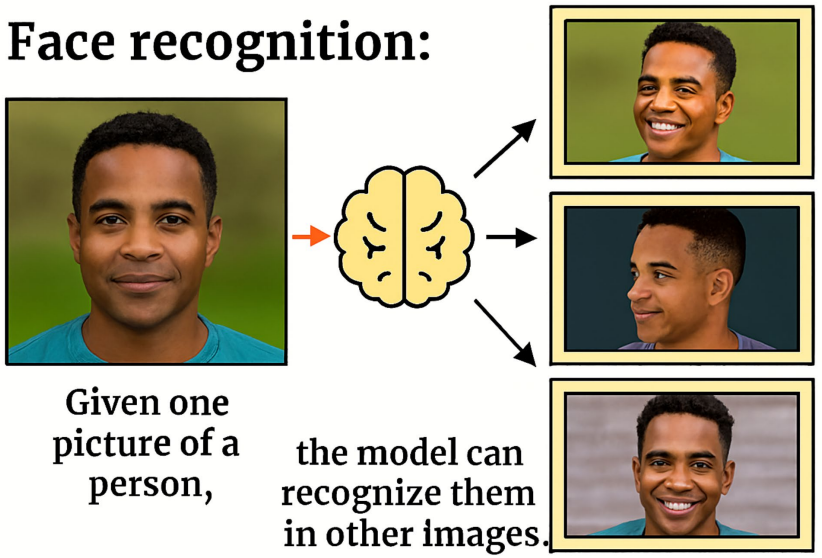
One-shot learning means the model can correctly recognize a new class after seeing just one example of it.

Example: Suppose the model has never seen a zebra before. If we show the model a single photo of a zebra, it should learn to generalize and correctly classify other zebra images.



Another **Example**:

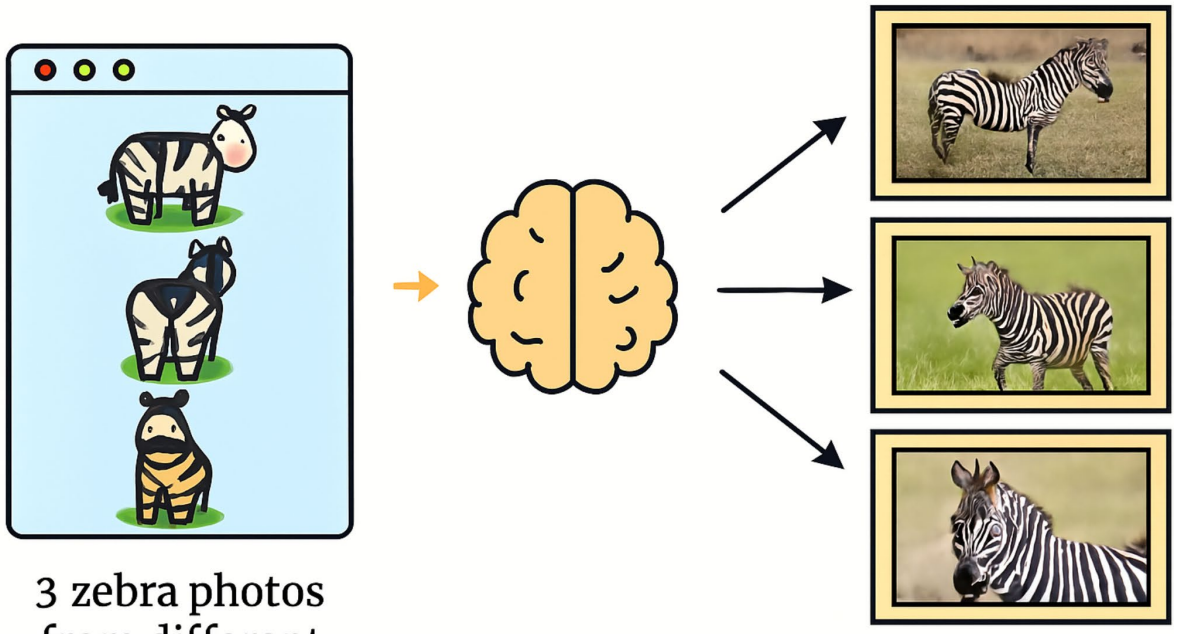
Face recognition: Given one picture of a person, the model can recognize them in other images.



Few-Shot Learning

In Few-shot learning, the model is given a few examples (e.g., 5–10 images) of the new class before making predictions.

Example: If we provide the model with 3 photos of zebras from different angles, it should then be able to identify zebras in new, unseen images.

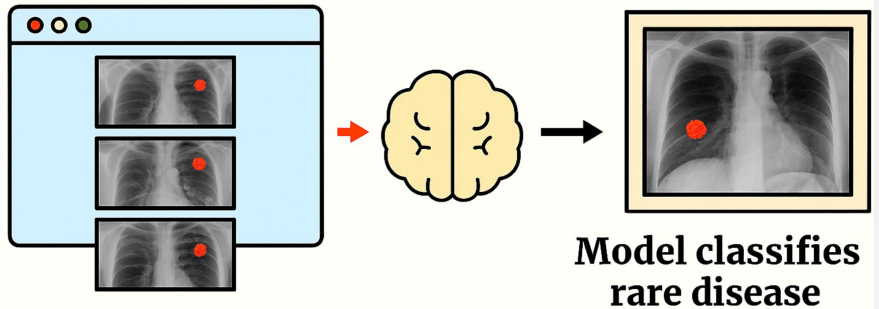


3 zebra photos from different angles provided to the model

Model able to identify zebras in new, unseen images

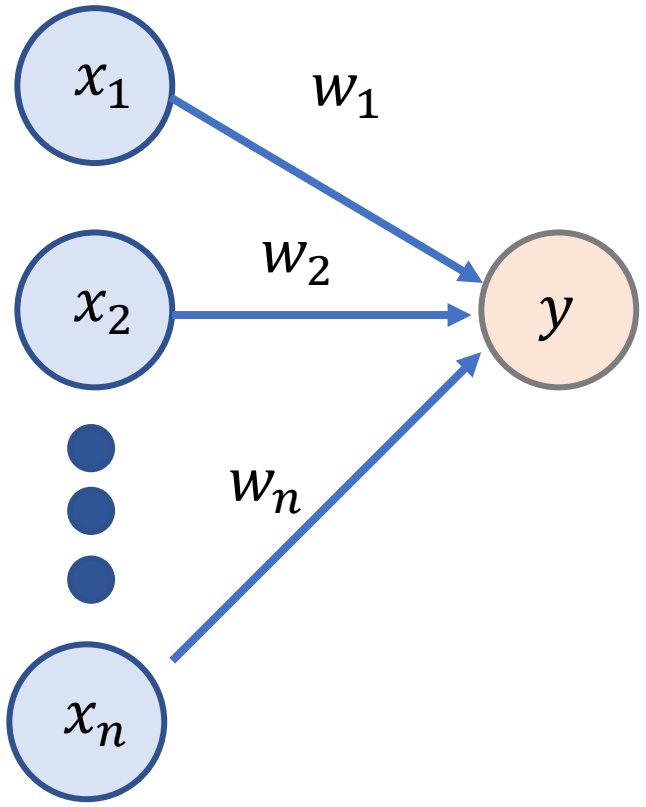
Additonal **Examples**:

- Medical imaging: Classifying rare diseases where only a handful of labeled cases exist.



- LLMs with prompts: If you give GPT a few examples in the prompt ("few-shot prompting"), it improves accuracy on the task.

Perceptron



1. Introduction

The **Perceptron** is one of the earliest models of an artificial neuron, introduced by Frank Rosenblatt in 1958. It is a simple binary linear classifier that inspired the development of modern neural networks.

2. Model Definition

Given an input vector

$$\mathbf{x} = (x_1, x_2, \ldots, x_n)^T \in \mathbb{R}^n,$$

the perceptron computes a weighted sum with bias:

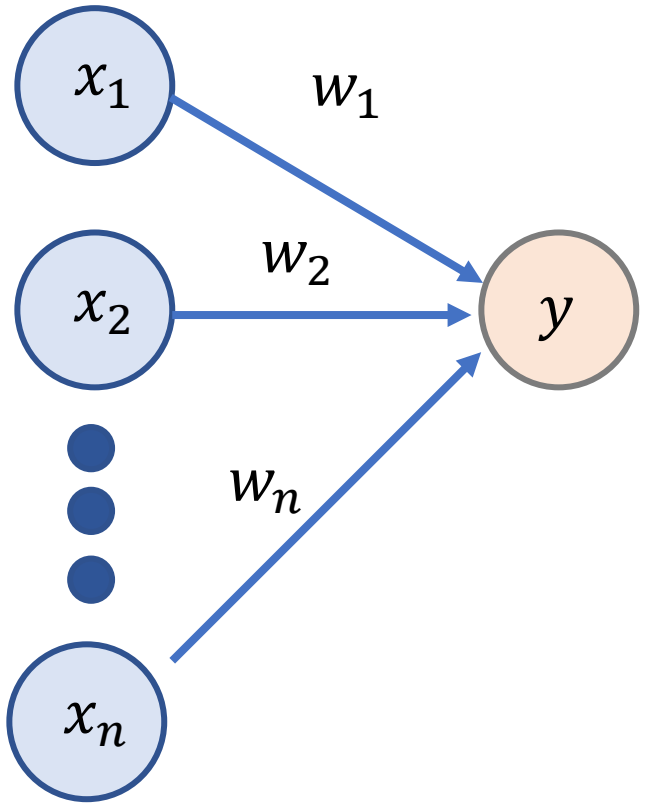
$$z = \mathbf{w}^T\mathbf{x} + b,$$

where $\mathbf{w} = (w_1, w_2, \ldots, w_n)^T$ is the weight vector and b is the bias term.

The perceptron output is obtained by applying a step activation function:

$$y = f(z) = \begin{cases} 1, & z \geq 0, \\ 0, & z < 0. \end{cases}$$

Perceptron



3. Learning Rule

The perceptron updates its weights iteratively to reduce classification errors. For a training sample $(\mathbf{x}, t)$ with target label $t \in \{0, 1\}$:

$$\mathbf{w} := \mathbf{w} + \eta(t - y)\mathbf{x},$$
$$b := b + \eta(t - y),$$

where $\eta > 0$ is the learning rate.

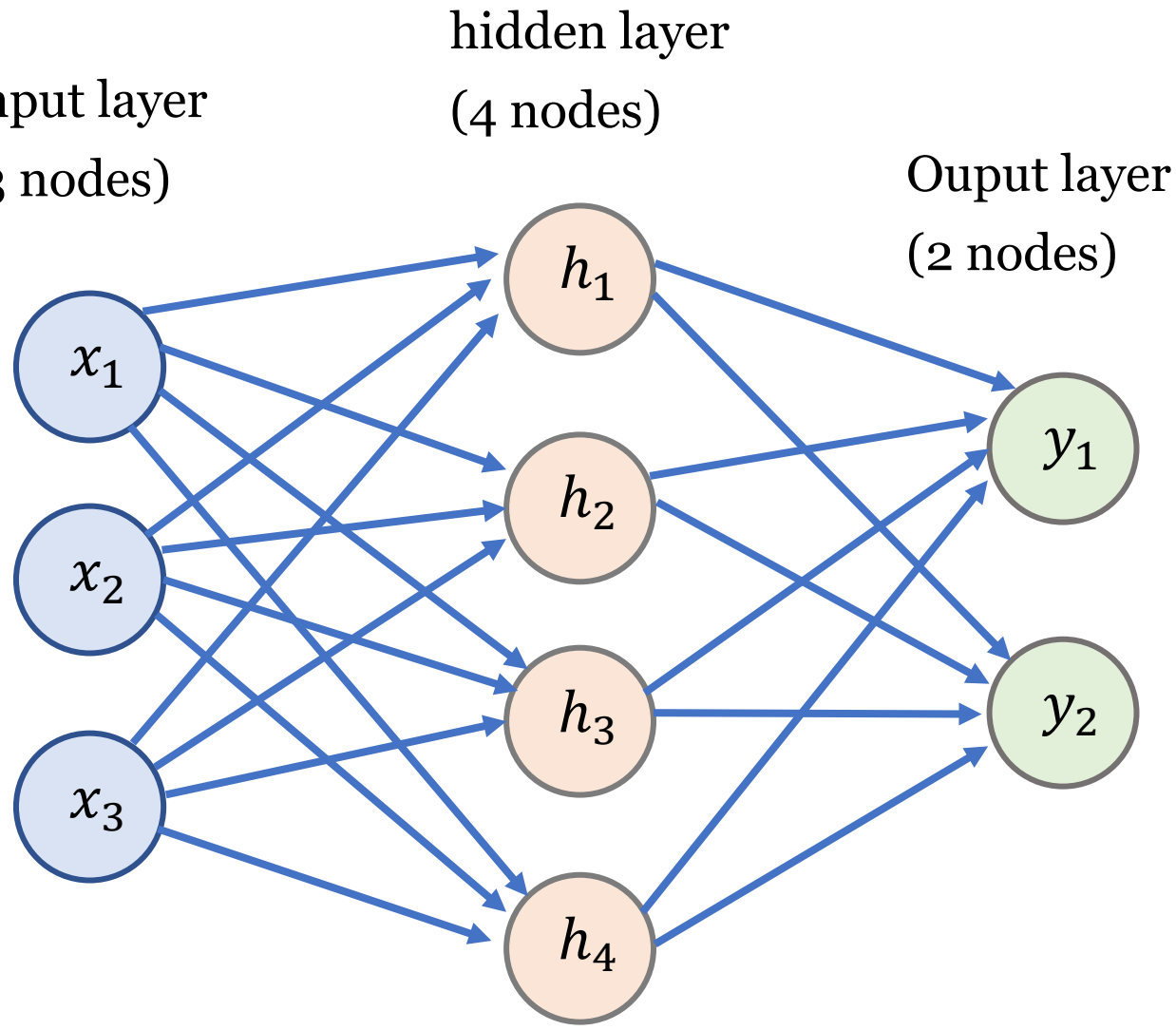
4. Properties

- The perceptron can classify linearly separable data.
- If the data is linearly separable, the perceptron learning algorithm converges in a finite number of steps.
- It cannot solve problems that are not linearly separable (e.g., the XOR problem).
- Despite its limitations, the perceptron was the foundation for modern neural network models.

5. Extension to Neural Networks

A single perceptron corresponds to one neuron. By stacking multiple perceptrons in layers and using differentiable activation functions (e.g., sigmoid, ReLU), we obtain modern multilayer neural networks capable of solving nonlinear problems.

Neural Network (e.g., classification)



Hidden Layer

For each hidden neuron h_j $(j = 1, 2, 3, 4)$:

$$z_j^{[1]} = w_{j1}^{[1]} x_1 + w_{j2}^{[1]} x_2 + w_{j3}^{[1]} x_3 + b_j^{[1]}$$

$$h_j = f\left(z_j^{[1]}\right)$$

where $f(\cdot)$ is an activation function (e.g., ReLU, sigmoid).

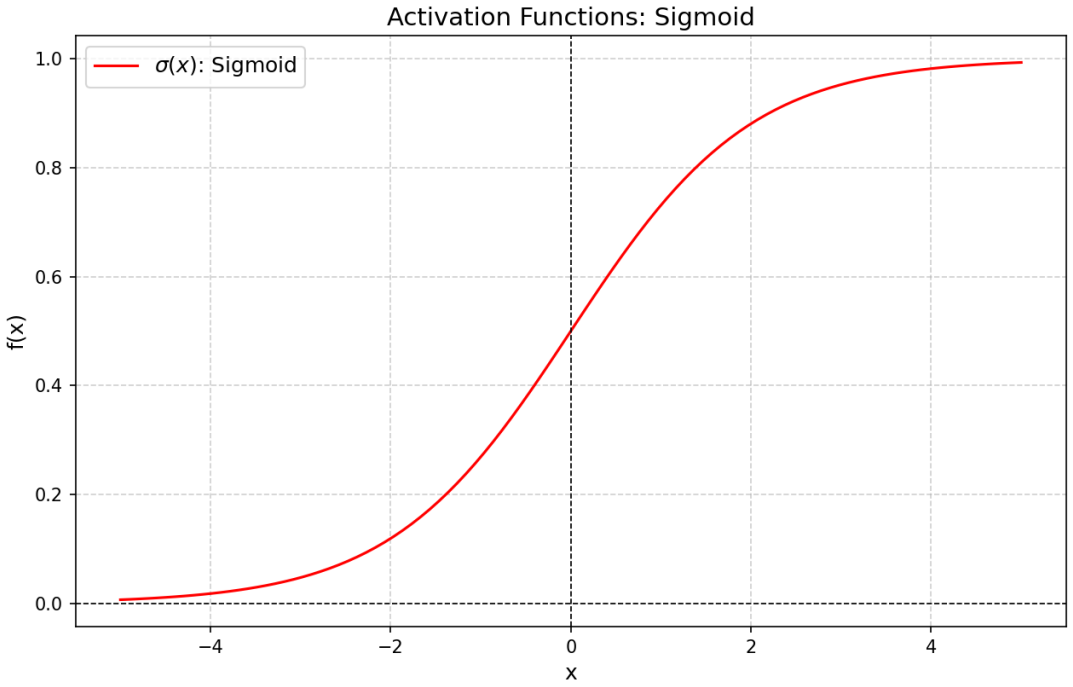
Activation Function (1): Sigmoid Function

```
import numpy as np
x = np.linspace(-5, 5, 100)
sigmoid = 1 / (1 + np.exp(-x))
```

Numpy

```
import torch
x = torch.linspace(-5, 5, 100)
sigmoid = torch.sigmoid(x)
```

PyTorch



$$f(x) = \frac{1}{1 + e^{-x}}$$

$$\lim_{x \to -\infty} f(x) = 0, \quad \lim_{x \to +\infty} f(x) = 1$$

- Maps any real number into the range $[0, 1]$.
- Smooth and differentiable everywhere.
- Saturates for very large positive or negative inputs, which can cause the vanishing gradient problem.

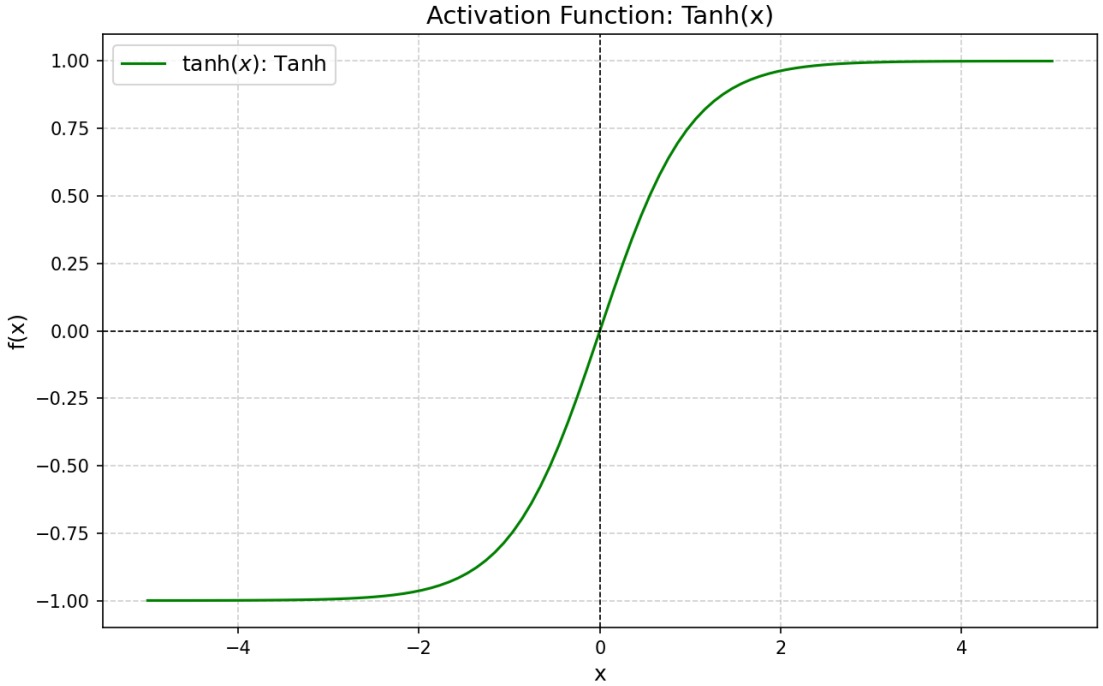
Activation Function (2): Tanh (Hyperbolic Tangent)

```
import numpy as np
x = np.linspace(-5, 5, 100)
tanh = np.tanh(x)
```

Numpy

```
import torch
x = torch.linspace(-5, 5, 100)
tanh = torch.tanh(x)
```

PyTorch



$$f(x) = \tanh(x) = \frac{e^x - e^{-x}}{e^x + e^{-x}}$$

$$\lim_{x \to -\infty} f(x) = -1, \quad \lim_{x \to +\infty} f(x) = 1$$

- Maps any real number into the range $[-1, 1]$.
- Zero-centered, unlike the sigmoid function.
- Still suffers from vanishing gradients for very large positive or negative inputs.

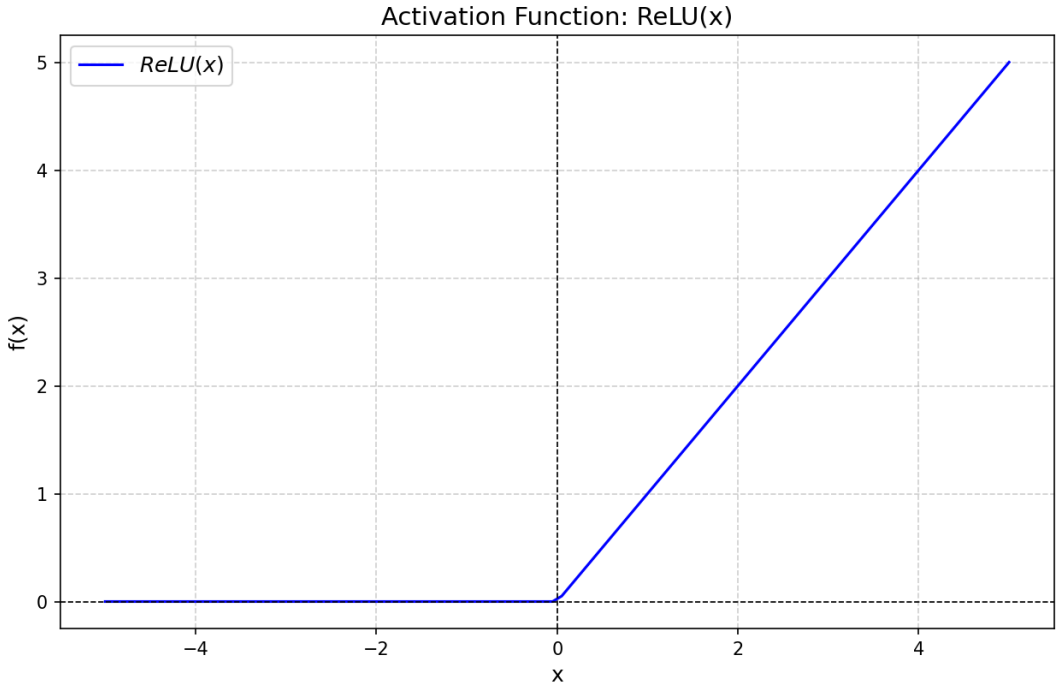
Activation Function (3): ReLU (Rectified Linear Unit)

```
import numpy as np
x = np.linspace(-5, 5, 100)
relu = np.maximum(0, x)
```

Numpy

```
import torch
x = torch.linspace(-5, 5, 100)
relu = torch.relu(x)
```

PyTorch



$$f(x) = \max(0, x)$$

$$f(x) = \begin{cases} x, & \text{if } x > 0 \\ 0, & \text{if } x \leq 0 \end{cases}$$

- Simple and computationally efficient.
- Helps avoid the vanishing gradient problem compared to sigmoid or tanh.
- Not differentiable at $x = 0$, but works well in practice.

Activation Function (4): GELU (Gaussian Error Linear Unit)

1) Definition

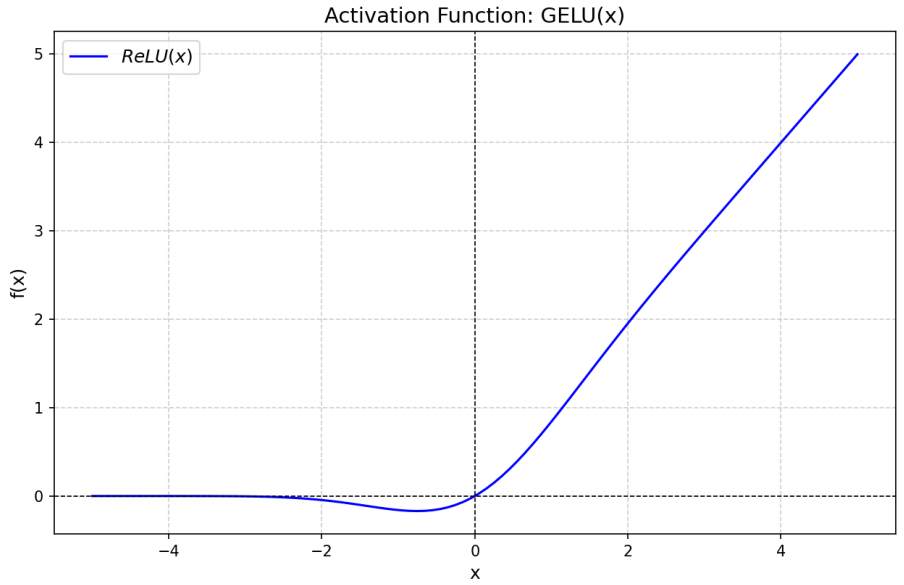
$$f(x) = x \cdot \Phi(x)$$

where $\Phi(x)$ is the cumulative distribution function (CDF) of the standard Gaussian distribution:

$$\Phi(x) = \frac{1}{2}\left(1 + \operatorname{erf}\left(\frac{x}{\sqrt{2}}\right)\right)$$

Thus, explicitly:

$$f(x) = \frac{x}{2}\left(1 + \operatorname{erf}\left(\frac{x}{\sqrt{2}}\right)\right)$$

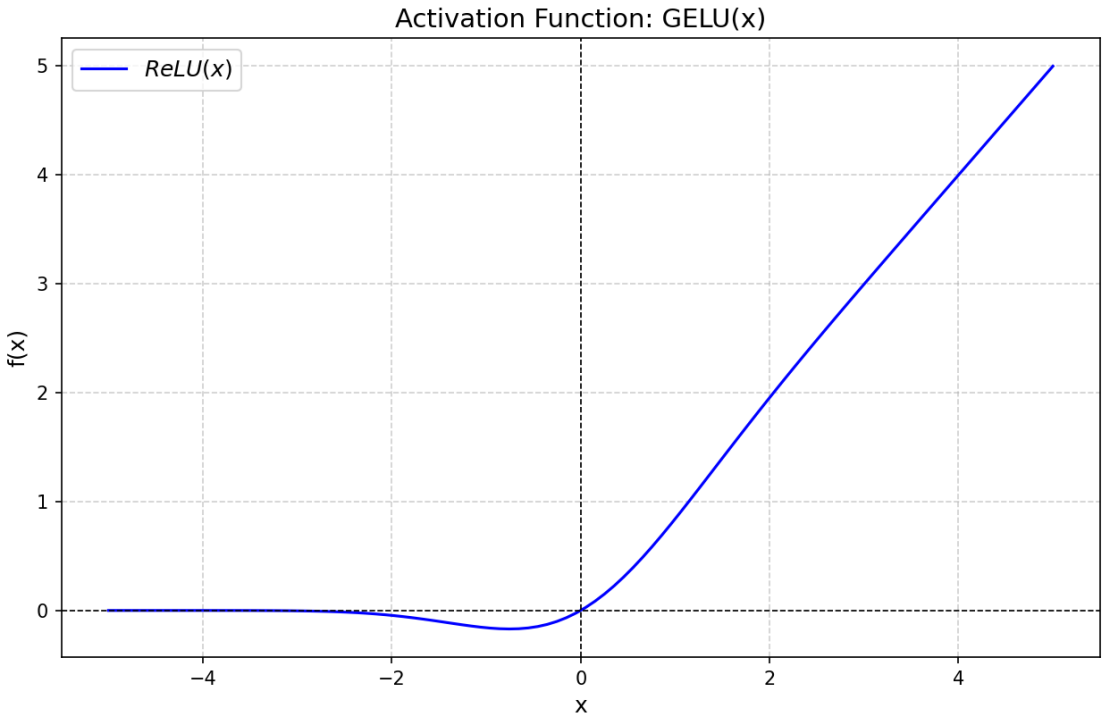


2) Approximation

Since computing the error function can be expensive, the following tanh-based approximation is often used:

$$f(x) \approx 0.5x\left(1 + \tanh\left(\sqrt{\frac{2}{\pi}}\left(x + 0.044715x^3\right)\right)\right)$$

Activation Function (4): GELU (Gaussian Error Linear Unit)



- Smooth and differentiable, unlike ReLU which has a sharp cutoff at zero.
- Probabilistically scales negative inputs instead of discarding them completely.
- Widely used in modern deep learning architectures such as BERT, GPT, and Vision Transformers.

Activation Function (4): GELU (Gaussian Error Linear Unit)

```python
import numpy as np
from math import sqrt, pi
from math import erf  # scalar version
# For vectorized erf, scipy.special.erf is recommended

def gelu_erf_numpy(x: np.ndarray) -> np.ndarray:
    # Exact GeLU: f(x) = 0.5 * x * (1 + erf(x / sqrt(2)))
    return 0.5 * x * (1.0 + np.vectorize(erf)(x / sqrt(2.0)))

def gelu_tanh_numpy(x: np.ndarray) -> np.ndarray:
    # Approximate GeLU:
    # f(x) ≈ 0.5 * x * (1 + tanh( sqrt(2/pi) * (x + 0.044715 * x^3) ))
    return 0.5 * x * (1.0 + np.tanh(np.sqrt(2.0 / np.pi) * (x + 0.044715 * x**3)))

# Example usage
x = np.linspace(-5, 5, 100)
y_exact = gelu_erf_numpy(x)
y_approx = gelu_tanh_numpy(x)
```

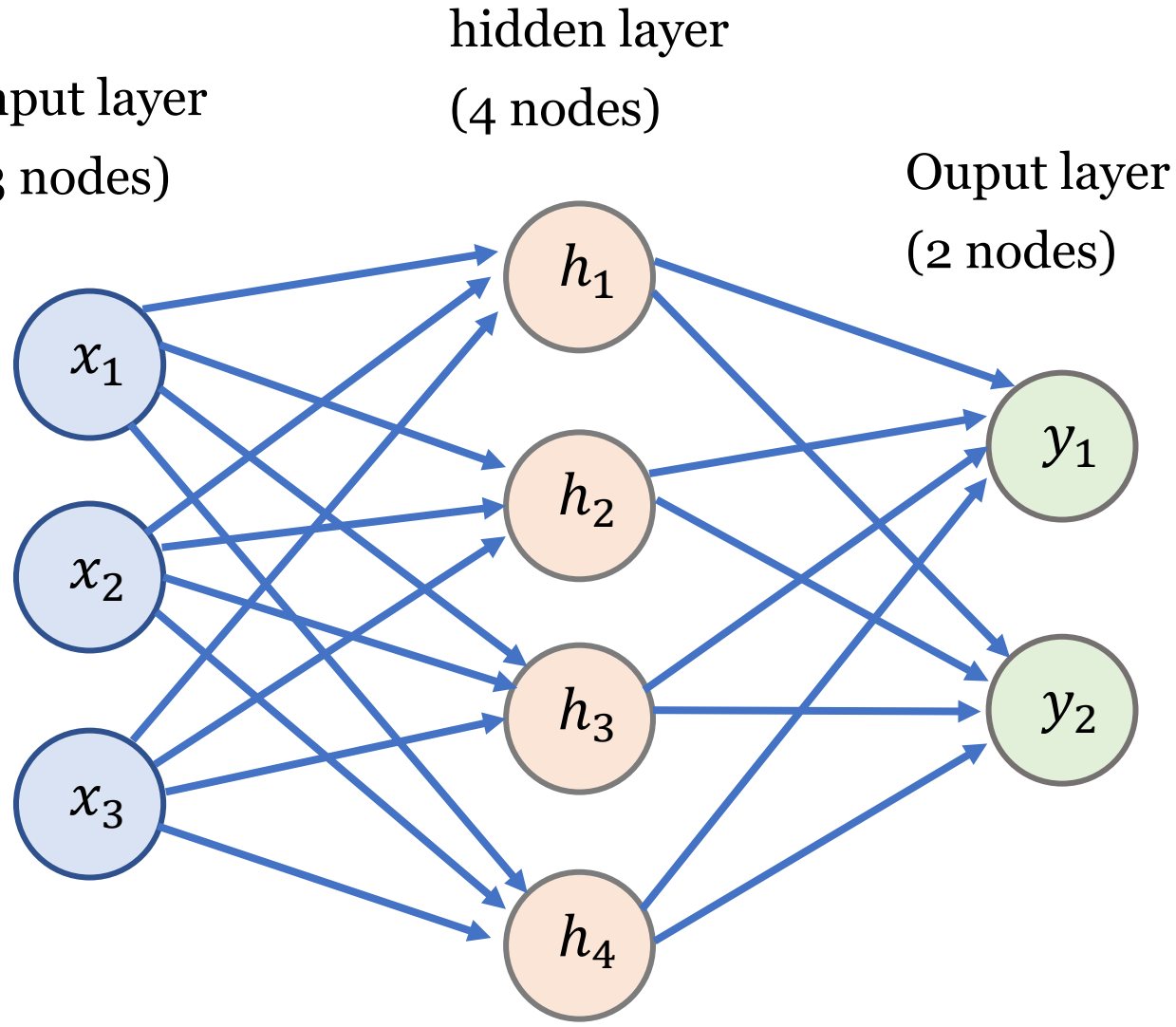
Numpy

Activation Function (4): GELU (Gaussian Error Linear Unit)

```
import torch
import torch.nn as nn
import torch.nn.functional as F

# Functional API
y_exact = F.gelu(x, approximate='none')       # Exact (ERF-based) version
y_approx = F.gelu(x, approximate='tanh')      # Tanh-based approximation

# Module API
gelu_exact = nn.GELU(approximate='none')
gelu_approx = nn.GELU(approximate='tanh')

y1 = gelu_exact(x)     # y_exact == gelu_exact
y2 = gelu_approx(x)    # y_approx == gelu_approx
```

PyTorch

Neural Network (e.g., classification)



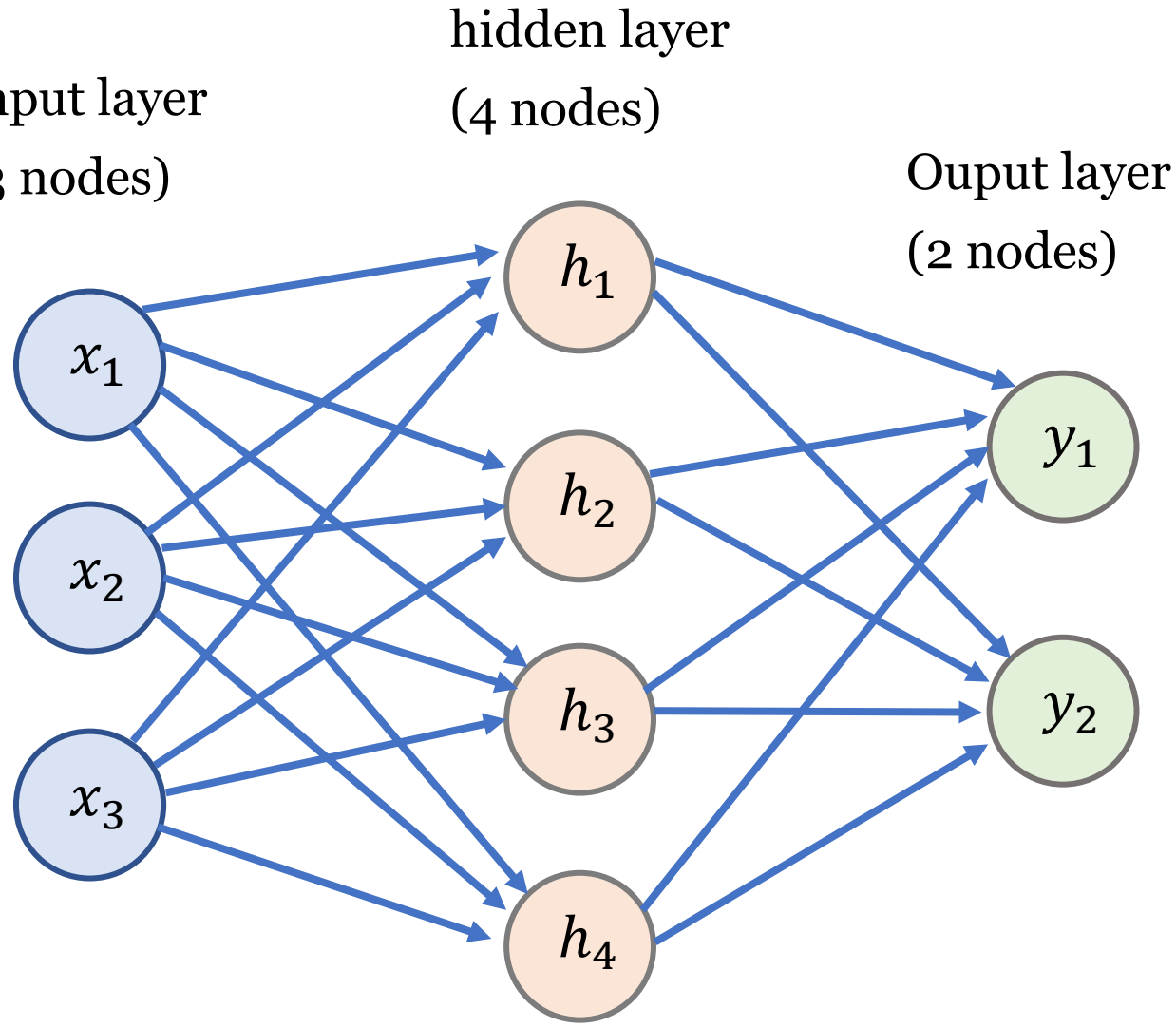
Output Layer

For each output neuron $\hat{y}_k$ $(k = 1, 2)$:

$$z_k^{[2]} = w_{k1}^{[2]} h_1 + w_{k2}^{[2]} h_2 + w_{k3}^{[2]} h_3 + w_{k4}^{[2]} h_4 + b_k^{[2]}$$

$$\hat{y}_k = \frac{e^{z_k^{[2]}}}{\sum_{l=1}^{2} e^{z_l^{[2]}}} \quad \text{(Softmax activation)}$$

Neural Network (e.g., classification)

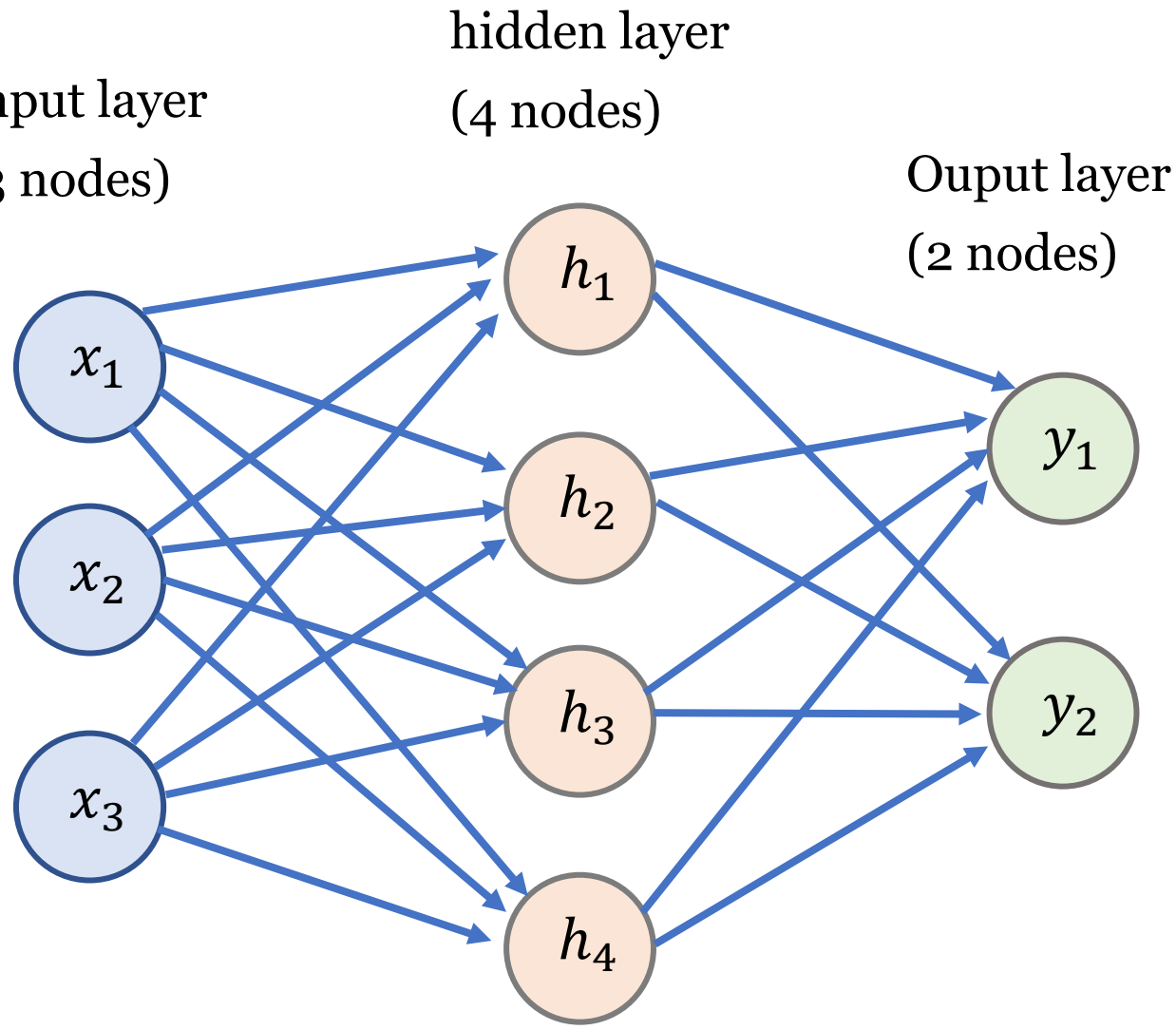


Compact Matrix Form

$$\mathbf{h} = f\left(W^{[1]}\mathbf{x} + \mathbf{b}^{[1]}\right), \quad \mathbf{x} \in \mathbb{R}^3,\ \mathbf{h} \in \mathbb{R}^4$$

$$\hat{\mathbf{y}} = \mathrm{Softmax}\left(W^{[2]}\mathbf{h} + \mathbf{b}^{[2]}\right), \quad \hat{\mathbf{y}} \in \mathbb{R}^2$$

Neural Network (e.g., classification)



Loss Function (Cross-Entropy)

For target label $y \in \{1, 2\}$:

$$L = -\sum_{k=1}^{2} \mathbf{1}\{y = k\} \log(\hat{y}_k)$$

where $\mathbf{1}\{y = k\}$ is an indicator function equal to 1 if the true class is k, otherwise 0.

Softmax Function

The **softmax function** converts a vector of real numbers into a probability distribution. Given an input vector

$$\mathbf{z} = (z_1, z_2, \ldots, z_K) \in \mathbb{R}^K,$$

the softmax function is defined as:

$$\sigma(\mathbf{z})_i = \frac{e^{z_i}}{\sum_{j=1}^{K} e^{z_j}}, \quad \text{for } i = 1, 2, \ldots, K.$$

- Each output value lies in the range $(0, 1)$.
- The outputs sum to 1:

$$\sum_{i=1}^{K} \sigma(\mathbf{z})_i = 1.$$

- Therefore, the softmax function transforms raw scores (logits) into a probability distribution over K classes.
- Softmax is commonly used in the final layer of a neural network for multi-class classification tasks.

Softmax Function

```python
import torch

logits = torch.tensor([[1.0, 2.0, 3.0],
                       [1.0, 2.0, 5.0]])

a = torch.softmax(logits, dim=0)  # column-wise
b = torch.softmax(logits, dim=1)  # row-wise

print(a)
print(a.sum(dim=0))  # should be all ones

print(b)
print(b.sum(dim=1))  # should be all ones
```

```
tensor([[0.5000, 0.5000, 0.1192],
        [0.5000, 0.5000, 0.8808]])
tensor([1.0000, 1.0000, 1.0000])
```

```
tensor([[0.0900, 0.2447, 0.6652],
        [0.0171, 0.0466, 0.9362]])
tensor([1., 1.])
```

```
a = torch.softmax(logits, dim=0)
→ normalizes down the rows, so each column sums to 1.

b = torch.softmax(logits, dim=1)
→ normalizes across the columns, so each row sums to 1.
```

Sigmoid Function in the output layer

- Binary classification ($K = 2$) $\rightarrow$ use **sigmoid**.
- Multi-class classification ($K > 2$) $\rightarrow$ use **softmax**.

1. Sigmoid Function

The sigmoid function is defined as:

$$\sigma(x) = \frac{1}{1 + e^{-x}}$$

It maps any real number into the range $(0, 1)$, which makes it suitable for binary classification.

2. Binary Classification (2 Classes)

For binary classification, the neural network produces a single logit z. Applying the sigmoid gives:

$$\hat{y} = \sigma(z) \in (0, 1)$$

Interpretation:

$$P(\text{Class } 1 \mid z) = \hat{y}, \quad P(\text{Class } 0 \mid z) = 1 - \hat{y}$$

The loss function is the **Binary Cross-Entropy (BCE)**:

$$L = -\Big(y \log(\hat{y}) + (1 - y) \log(1 - \hat{y})\Big), \quad y \in \{0, 1\}$$

torch.nn.Linear ( )

```python
import torch
import torch.nn as nn

# Create a linear layer: y = xW + b
layer = nn.Linear(in_features=4, out_features=2)

# Random input tensor with shape (batch_size=3, in_features=4)
x = torch.randn(3, 4)

# Call the layer like a function
y = layer(x)

print("Input shape:", x.shape)
print("Output shape:", y.shape)
```

```
Input shape: torch.Size([3, 4])
Output shape: torch.Size([3, 2])
```

`torch.nn.Linear` are callable in PyTorch.

This is because all subclasses of torch.nn.Module implement the special method `__call__`, which internally calls the forward method.

- `nn.Linear` is a class → you must instantiate it (e.g., `layer = nn.Linear(4, 2)`).
- The instance (layer) is callable like a function because it inherits from nn.Module.
- `layer.__call__()` → does pre/post processing → calls `layer.forward()`.

Initialization of Weights and Biases

1. Purpose of Initialization

When training deep neural networks:

- If weights are too **large**, activations and gradients may **explode**.
- If weights are too **small**, activations and gradients may **vanish**.

Good initialization tries to keep the **variance of activations and gradients stable** across layers.

2. Target Variance

We define a **target variance** v for the weights:

$$\mathrm{Var}(W) = v.$$

The value of v depends on the chosen initialization scheme:

- For **Kaiming (He)** initialization (ReLU),

$$v = \frac{2}{\text{fan\_in}}.$$

- For **Xavier (Glorot)** initialization,

$$v = \frac{2}{\text{fan\_in} + \text{fan\_out}}.$$

All parameter derivations (uniform bound, Gaussian variance) aim to enforce this condition.

3. Why a Bound or Variance is Used

There are two common sampling strategies:

- **Uniform distribution:**

$$W \sim \mathcal{U}(-\text{bound}, +\text{bound}),$$

  where the *bound* is chosen so that $\text{Var}(W) = v$.

- **Normal (Gaussian) distribution:**

$$W \sim \mathcal{N}(0, \sigma^2),$$

  where the variance σ^2 is directly set to v.

4. Example: Variance of Uniform vs Normal

Uniform distribution. For $X \sim U(-a, a)$:

$$\text{Var}(X) = \frac{a^2}{3}.$$

Setting $\text{Var}(X) = v$ gives

$$a = \sqrt{3v}.$$

Normal distribution. For $X \sim \mathcal{N}(0, \sigma^2)$:

$$\text{Var}(X) = \sigma^2.$$

So we directly set

$$\sigma^2 = v.$$

5. In Kaiming and Xavier Initialization (Weights)

- **Kaiming (He) initialization (ReLU case):**

$$v = \frac{2}{\text{fan\_in}}.$$

- Uniform form:

$$\text{bound} = \sqrt{\frac{6}{\text{fan\_in}}}.$$

- Normal form:

$$\sigma^2 = \frac{2}{\text{fan\_in}}.$$

- **Xavier (Glorot) initialization:**

$$v = \frac{2}{\text{fan\_in} + \text{fan\_out}}.$$

- Uniform form:

$$\text{bound} = \sqrt{\frac{6}{\text{fan\_in}+\text{fan\_out}}}.$$

- Normal form:

$$\sigma^2 = \frac{2}{\text{fan\_in} + \text{fan\_out}}.$$

6. Bias Initialization in PyTorch

In contrast to weights, PyTorch initializes biases using a simpler rule:

$$b \sim \mathcal{U}\left(-\frac{1}{\sqrt{\text{fan\_in}}}, +\frac{1}{\sqrt{\text{fan\_in}}}\right).$$

This does not aim to preserve variance, but simply keeps bias values small relative to the number of input connections.

- **Kaiming (He) Initialization**
  - Default for `nn.Linear` in PyTorch.
  - Uses only fan_in.
  - Best for ReLU-like activations.
- **Xavier (Glorot) Initialization**
  - Must be applied manually with `torch.nn.init`.
  - Uses both fan_in and fan_out.
  - Best for sigmoid/tanh activations.

7. PyTorch Example (Conceptual)

```
import torch, math

in_features, out_features = 3, 2
fan_in, fan_out = in_features, out_features

# Target variance v
v_kaiming = 2.0 / fan_in
v_xavier  = 2.0 / (fan_in + fan_out)

# Kaiming initialization
bound_kaiming = math.sqrt(3 * v_kaiming)          # uniform
sigma_kaiming = math.sqrt(v_kaiming)              # normal

# Xavier initialization
bound_xavier = math.sqrt(3 * v_xavier)            # uniform
sigma_xavier = math.sqrt(v_xavier)                # normal

# Bias bound
bound_bias = 1.0 / math.sqrt(fan_in)

# Generate weights and bias
w_kaiming_u = torch.empty(out_features, in_features).uniform_(-bound_kaiming, bound_kaiming)
w_kaiming_n = torch.empty(out_features, in_features).normal_(0.0, sigma_kaiming)

w_xavier_u  = torch.empty(out_features, in_features).uniform_(-bound_xavier, bound_xavier)
w_xavier_n  = torch.empty(out_features, in_features).normal_(0.0, sigma_xavier)

b_init      = torch.empty(out_features).uniform_(-bound_bias, bound_bias)
```

```
import torch
import torch.nn as nn

layer = nn.Linear(3, 2)

print(layer.weight)
print(layer.bias)
```

Check Kaiming and Xavier Initialization

```
import torch
import torch.nn as nn
import torch.nn.init as init
import math

# Define a linear layer
layer = nn.Linear(in_features=3, out_features=2, bias=True)

# --- Default behavior ---
# By default, nn.Linear uses Kaiming uniform initialization for weights
# and uniform(-1/sqrt(fan_in), 1/sqrt(fan_in)) for bias.
print("Default (Kaiming) weight:\n", layer.weight)
print("Default bias:\n", layer.bias)

# --- Xavier initialization (optional) ---
# If you prefer Xavier, reinitialize explicitly:
init.xavier_uniform_(layer.weight)
# The trailing underscore _ in PyTorch convention means
# in-place operation (it modifies the tensor directly)

fan_in = layer.in_features
bound_bias = 1 / math.sqrt(fan_in)

if layer.bias is not None:
    layer.bias.data.uniform_(-bound_bias, bound_bias)

print("\nXavier-initialized weight:\n", layer.weight)
print("Xavier-initialized bias:\n", layer.bias)
```

```
Default (Kaiming) weight:
 Parameter containing:
tensor([[-0.0480,  0.2426, -0.2729],
        [ 0.5431,  0.3103,  0.0399]], requires_grad=True)
Default bias:
 Parameter containing:
tensor([0.3220, 0.4163], requires_grad=True)

Xavier-initialized weight:
 Parameter containing:
tensor([[ 0.7014,  0.7367,  0.1130],
        [-0.4929, -0.0240, -0.5067]], requires_grad=True)
Xavier-initialized bias:
 Parameter containing:
tensor([ 0.0247, -0.2385], requires_grad=True)
```

Backpropagation

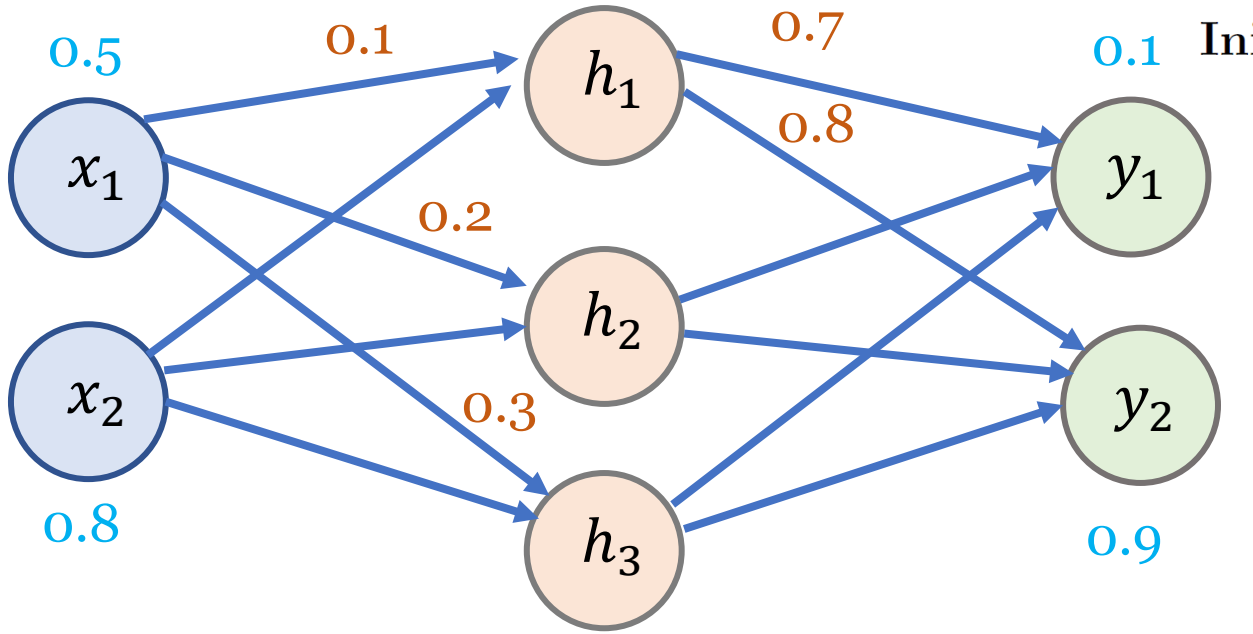
Linear Regression

Input layer (2 nodes)

hidden layer (3 nodes)

Ouput layer (2 nodes)

Activation Functions: **ReLU**



Initial Network State (explicit values)

- **Input (row vector):** $x = \begin{pmatrix} 0.5 & 0.8 \end{pmatrix}$
- **Target (row vector):** $t = \begin{pmatrix} 0.1 & 0.9 \end{pmatrix}$
- **Weights:** $W_{ih} = \begin{pmatrix} 0.1 & 0.2 & 0.3 \\ 0.4 & 0.5 & 0.6 \end{pmatrix}$, $W_{ho} = \begin{pmatrix} 0.7 & 0.8 \\ 0.9 & 0.1 \\ 0.2 & 0.3 \end{pmatrix}$
- **Biases (row vectors):** $B_h = \begin{pmatrix} 0.2 & 0.4 & 0.6 \end{pmatrix}$, $B_o = \begin{pmatrix} 0.5 & 0.7 \end{pmatrix}$
- **Loss (MSE):** $E = \frac{1}{2} \sum_{i=1}^{2} (t_i - o_i)^2$.

1. Forward Pass (row-vector convention)

$$\text{net}_h = x\, W_{ih} + B_h = \begin{pmatrix} 0.57 & 0.90 & 1.23 \end{pmatrix},$$
$$\text{out}_h = \text{ReLU}(\text{net}_h) = \begin{pmatrix} 0.57 & 0.90 & 1.23 \end{pmatrix}$$

$$\text{net}_o = \text{out}_h\, W_{ho} + B_o = \begin{pmatrix} 1.955 & 1.615 \end{pmatrix},$$
$$\text{out}_o = \text{ReLU}(\text{net}_o) = \begin{pmatrix} 1.955 & 1.615 \end{pmatrix},$$

$$E_{\text{total}} = \frac{1}{2}(0.1 - 1.955)^2 + \frac{1}{2}(0.9 - 1.615)^2 \approx 2.07.$$

Backpropagation

Linear Regression

Activation Functions: **ReLU**

2. Backward Pass (local × upstream)

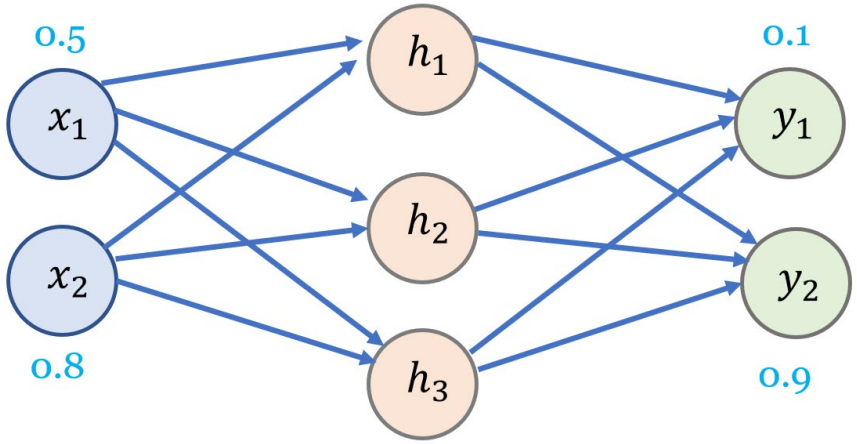
2.1. Output layer (define the output error signal). For MSE,

$$\frac{\partial E}{\partial \text{out}_o} = \text{out}_o - t = \begin{pmatrix} 1.855 & 0.715 \end{pmatrix}.$$

Since $\text{net}_o > 0$, $\text{ReLU}'(\text{net}_o) = \begin{pmatrix} 1 & 1 \end{pmatrix}$. Define

$$q_o := \underbrace{\text{ReLU}'(\text{net}_o)}_{\text{local}} \odot \underbrace{\frac{\partial E}{\partial \text{out}_o}}_{\text{upstream}} = \begin{pmatrix} 1.855 & 0.715 \end{pmatrix} \quad \Rightarrow \quad \frac{\partial E}{\partial \text{net}_o} = q_o.$$

where $\odot$ is the element-wise multiplication.



Backpropagation

Linear Regression

Activation Functions: **ReLU**

2.2. Hidden-to-output weights and biases (reuse q_o). Using $\text{net}_o = \text{out}_h W_{ho} + B_o$,

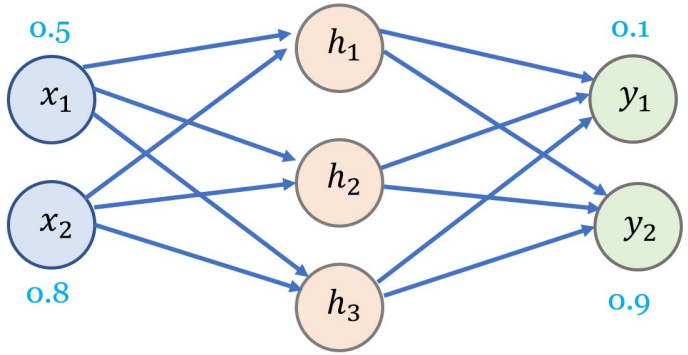
$$\frac{\partial E}{\partial W_{ho}} = \underbrace{\text{out}_h^T}_{\text{local}} \underbrace{q_o}_{\text{upstream}} = \begin{pmatrix} 1.05735 & 0.40755 \\ 1.66950 & 0.64350 \\ 2.28165 & 0.87945 \end{pmatrix}, \qquad \frac{\partial E}{\partial B_o} = \underbrace{1}_{\text{local}} \underbrace{q_o}_{\text{upstream}} = \begin{pmatrix} 1.855 & 0.715 \end{pmatrix}$$

2.3. Hidden layer (define the hidden error signal). Upstream to hidden outputs:

$$\frac{\partial E}{\partial \text{out}_h} = q_o \, W_{ho}^T = \begin{pmatrix} 1.8705 & 1.741 & 0.5855 \end{pmatrix}.$$

Since $\text{net}_h > 0$, $\text{ReLU}'(\text{net}_h) = \begin{pmatrix} 1 & 1 & 1 \end{pmatrix}$. Define

$$q_h := \underbrace{\text{ReLU}'(\text{net}_h)}_{\text{local}} \odot \underbrace{\left(\frac{\partial E}{\partial \text{out}_h} \right)}_{\text{upstream}} = \begin{pmatrix} 1.8705 & 1.741 & 0.5855 \end{pmatrix} \quad \Rightarrow \quad \frac{\partial E}{\partial \text{net}_h} = q_h.$$



Backpropagation

Linear Regression

Activation Functions: **ReLU**

2.4. Input-to-hidden weights and biases (reuse q_h). From $\text{net}_h = xW_{ih} + B_h$,

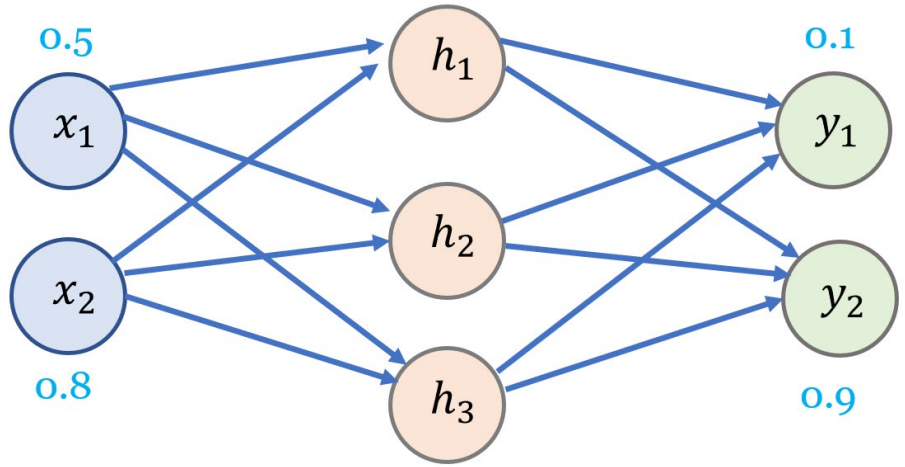
$$\frac{\partial E}{\partial W_{ih}} = \underbrace{x^T}_{\text{local}} \underbrace{q_h}_{\text{upstream}} = \begin{pmatrix} 0.93525 & 0.8705 & 0.29275 \\ 1.4964 & 1.3928 & 0.4684 \end{pmatrix}, \qquad \frac{\partial E}{\partial B_h} = \underbrace{1}_{\text{local}} \underbrace{q_h}_{\text{upstream}} = \begin{pmatrix} 1.8705 & 1.741 & 0.5855 \end{pmatrix}$$

3. Update step

Gradient descent updates (learning rate η):

$$W_{ho} \leftarrow W_{ho} - \eta \frac{\partial E}{\partial W_{ho}}, \quad B_o \leftarrow B_o - \eta \frac{\partial E}{\partial B_o},$$

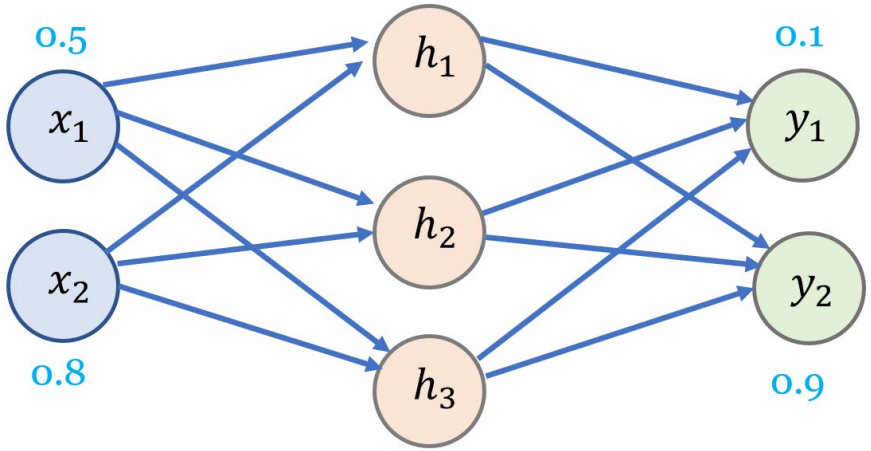
$$W_{ih} \leftarrow W_{ih} - \eta \frac{\partial E}{\partial W_{ih}}, \quad B_h \leftarrow B_h - \eta \frac{\partial E}{\partial B_h}.$$



Backpropagation – Linear Regression (2-3-2)

c03_backprop_linear_reg_ReLU_242.py

```
import torch
import torch.nn as nn
import torch.optim as optim
import matplotlib.pyplot as plt

# ---- 1) Model ----
class myModel(nn.Module):
    def __init__(self):
        super().__init__()
        self.fc1 = nn.Linear(2, 3, bias=True)  # input(2) -> hidden(3)
        self.fc2 = nn.Linear(3, 2, bias=True)  # hidden(3) -> output(2)
        self.reset_to_given_values()

    def reset_to_given_values(self):
        # Given matrices in the document
        W_ih = torch.tensor([[0.1, 0.2, 0.3],
                             [0.4, 0.5, 0.6]], dtype=torch.float32)
        B_h  = torch.tensor([0.2, 0.4, 0.6], dtype=torch.float32)
        W_ho = torch.tensor([[0.7, 0.8],
                             [0.9, 0.1],
                             [0.2, 0.3]], dtype=torch.float32)
        B_o  = torch.tensor([0.5, 0.7], dtype=torch.float32)

        with torch.no_grad():
            self.fc1.weight.copy_(W_ih.t())  # (3,2)
            self.fc1.bias.copy_(B_h)
            self.fc2.weight.copy_(W_ho.t())  # (2,3)
            self.fc2.bias.copy_(B_o)

    def forward(self, x):
        x = torch.relu(self.fc1(x))  # hidden layer only
        x = self.fc2(x)              # output layer: no ReLU
        return x
```



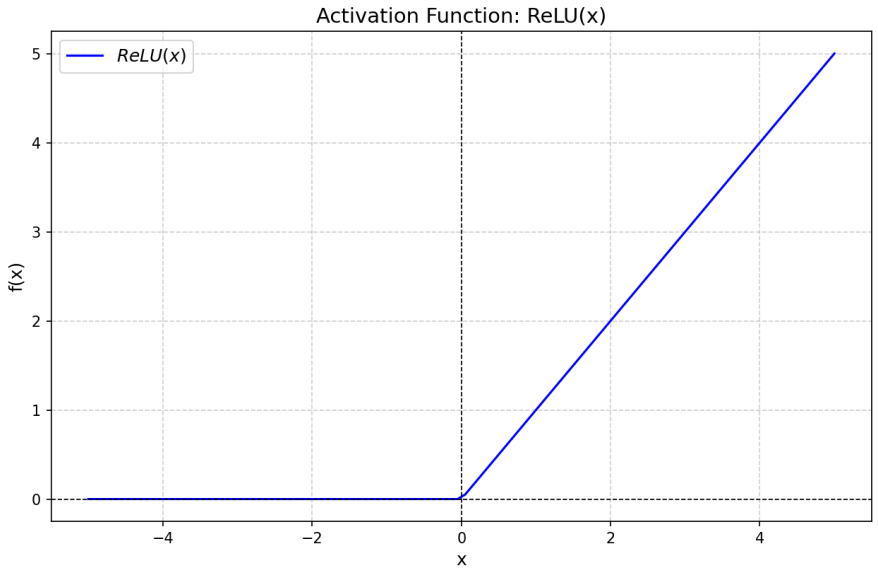
IMPORTANT

Remove the ReLU from the output layer

- When you used the **default PyTorch initialization**, it's possible that the **pre-activation outputs of the final layer were negative**.
- If they are negative, the output layer's ReLU sets them to zero, and the gradient of ReLU for negative inputs is also zero.
- Result: those output units become "dead" → no gradient flows → weights don't update → the loss does not decrease.

$$f(x) = \max(0, x)$$

$$f(x) = \begin{cases} x, & \text{if } x > 0 \\ 0, & \text{if } x \leq 0 \end{cases}$$

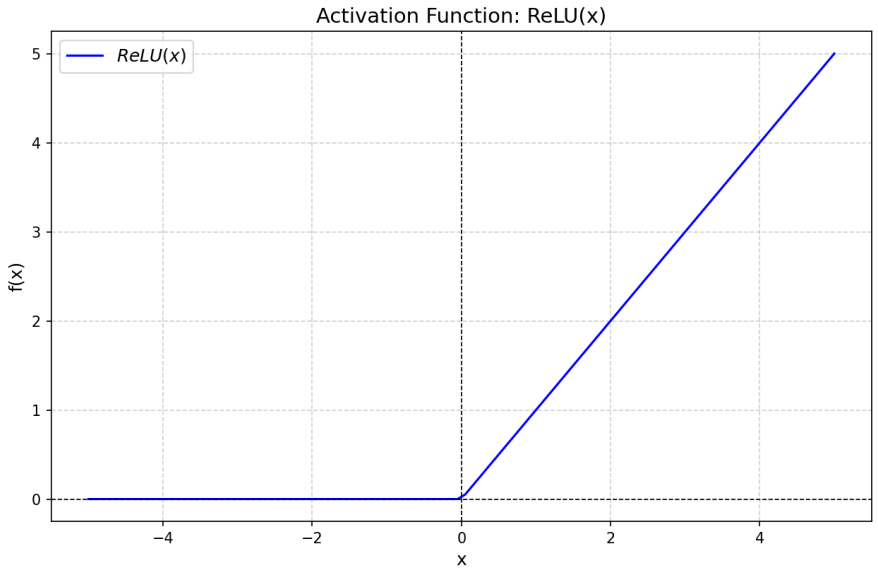


Remove the ReLU from the output layer

- For regression tasks, it's still best practice to remove the ReLU from the output layer.
- But if your initialization (i.e., lucky case) happens to place outputs in the ReLU's active region, training can still succeed.

$$f(x) = \max(0, x)$$

$$f(x) = \begin{cases} x, & \text{if } x > 0 \\ 0, & \text{if } x \leq 0 \end{cases}$$



How forward( ) is called in PyTorch

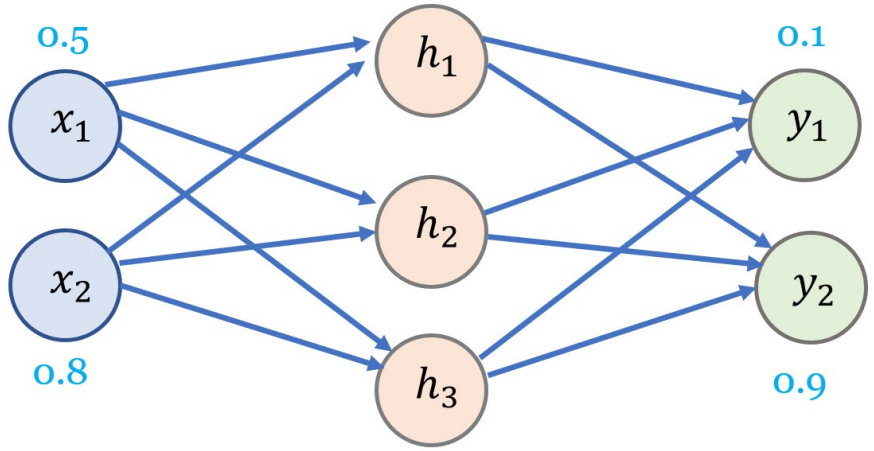
- In PyTorch, you don't usually call `forward()` directly.

```
import torch
import torch.nn as nn

class MyModel(nn.Module):
    def forward(self, x):
        print('hello')
        return x

x = torch.tensor([1.0])

net = MyModel()         # create an instance
y = net(x)              # calls forward(x)
print(y)
```

- `MyModel()` triggers the __call__ method defined in the base class **nn.Module**.
- Inside __call__, PyTorch does several things automatically:
  - ✓ Prepares the input for autograd (automatic differentiation).
  - ✓ Calls your model's `forward( )` method with the input.
  - ✓ Processes the output and attaches the computation graph for backpropagation.
- So, in effect, model(x) → calls forward(x) behind the scenes.

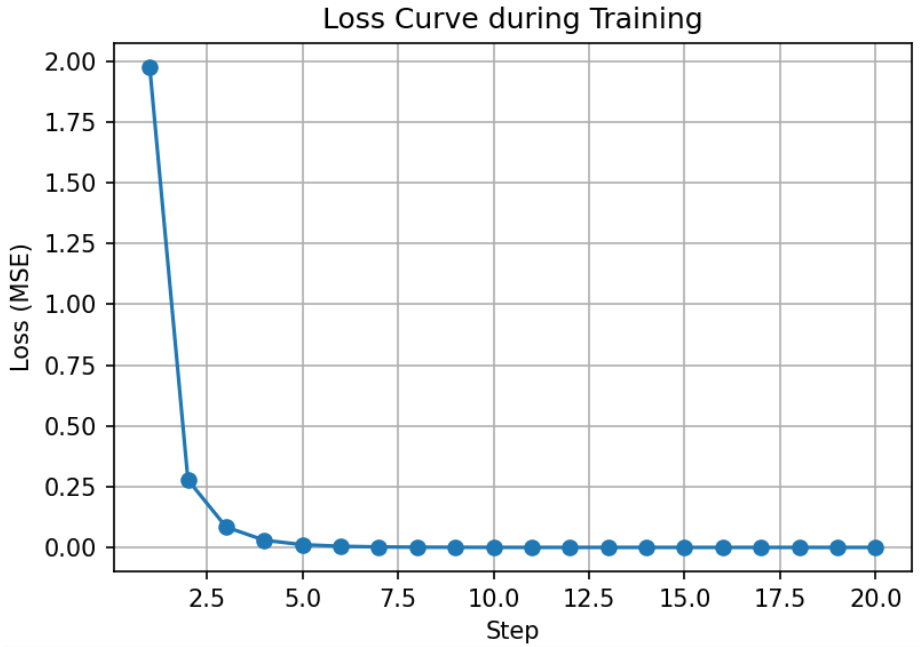
Backpropagation – Linear Regression (2-3-2)

```
# ---- 2) Data / Loss / Optimizer ----
model = myModel()
x = torch.tensor([0.5, 0.8], dtype=torch.float32)
t = torch.tensor([0.1, 0.9], dtype=torch.float32)

criterion = nn.MSELoss(reduction='mean')  # default MSE
learning_rate = 0.10
optimizer = optim.SGD(model.parameters(), lr=learning_rate)

# ---- 3) print parameters ----
def print_params(step, mdl):
    with torch.no_grad():
        Wih = mdl.fc1.weight.t()
        Bh  = mdl.fc1.bias
        Who = mdl.fc2.weight.t()
        Bo  = mdl.fc2.bias
        print(f"\n[After step {step}]")
        print("W_ih =\n", Wih.numpy())
        print("B_h  = ", Bh.numpy())
        print("W_ho =\n", Who.numpy())
        print("B_o  = ", Bo.numpy())
```



Backpropagation – Linear Regression (2-3-2)

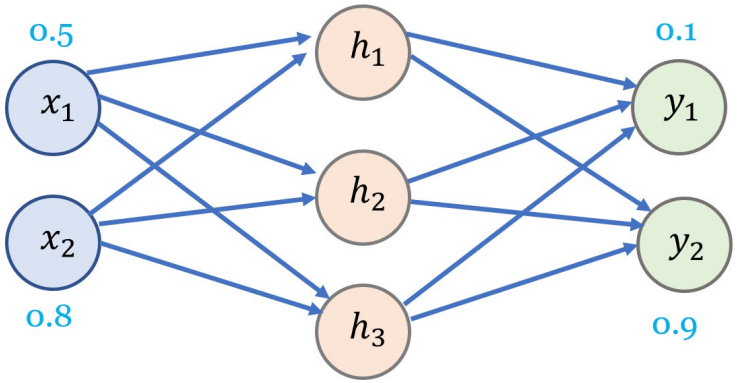
```
# ---- 4) Training loop with loss tracking ----
losses = []

for step in range(1, 21):  # 20 steps to see the curve
    optimizer.zero_grad()
    y = model(x)
    loss = criterion(y, t)
    loss.backward()
    optimizer.step()

    losses.append(loss.item())  # record loss
    print(f"step {step} | loss = {loss.item():.6f}")

    if step % 5 == 0:
        print_params(step, model)

# ---- 5) Plot loss curve ----
plt.figure(figsize=(6,4))
plt.plot(range(1, len(losses)+1), losses, marker='o')
plt.xlabel("Step")
plt.ylabel("Loss (MSE)")
plt.title("Loss Curve during Training")
plt.grid(True)
plt.show()
```




```
[After step 5]
W_ih =
 [[-0.04336868  0.05882919  0.27575284]
 [ 0.17061011  0.2741267   0.5612046 ]]
B_h  =  [-0.08673736  0.11765838  0.5515057 ]
W_ho =
 [[ 0.571718    0.7558604 ]
 [ 0.6565286   0.02737943]
 [-0.19934106  0.19679347]]
B_o  =  [0.16177835 0.6148201 ]
step 6 | loss = 0.004628
step 7 | loss = 0.002013
step 8 | loss = 0.000912
step 9 | loss = 0.000425
step 10 | loss = 0.000202
```

Compact code for Linear Regression (2-3-2)

```
import torch
import torch.nn as nn
import torch.optim as optim
import matplotlib.pyplot as plt

# ---- 1) Model ----
class MyModel(nn.Module):
    def __init__(self):
        super().__init__()
        self.fc1 = nn.Linear(2, 3)  # input(2) -> hidden(3)
        self.fc2 = nn.Linear(3, 2)  # hidden(3) -> output(2)

    def forward(self, x):
        x = torch.relu(self.fc1(x))  # ReLU only in the hidden layer
        x = self.fc2(x)              # No activation on the output layer
        return x

# ---- 2) Data / Loss / Optimizer ----
model = MyModel()
x = torch.tensor([0.5, 0.8], dtype=torch.float32).unsqueeze(0)  # [1, 2]
t = torch.tensor([0.1, 0.9], dtype=torch.float32).unsqueeze(0)  # [1, 2]

criterion = nn.MSELoss()
optimizer = optim.SGD(model.parameters(), lr=0.01)  # smaller LR for stability
```

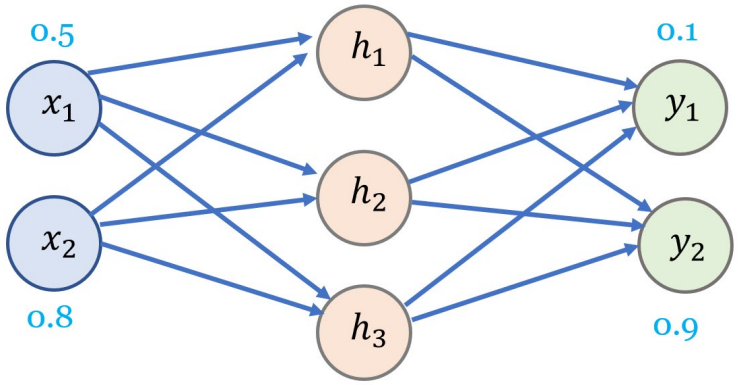


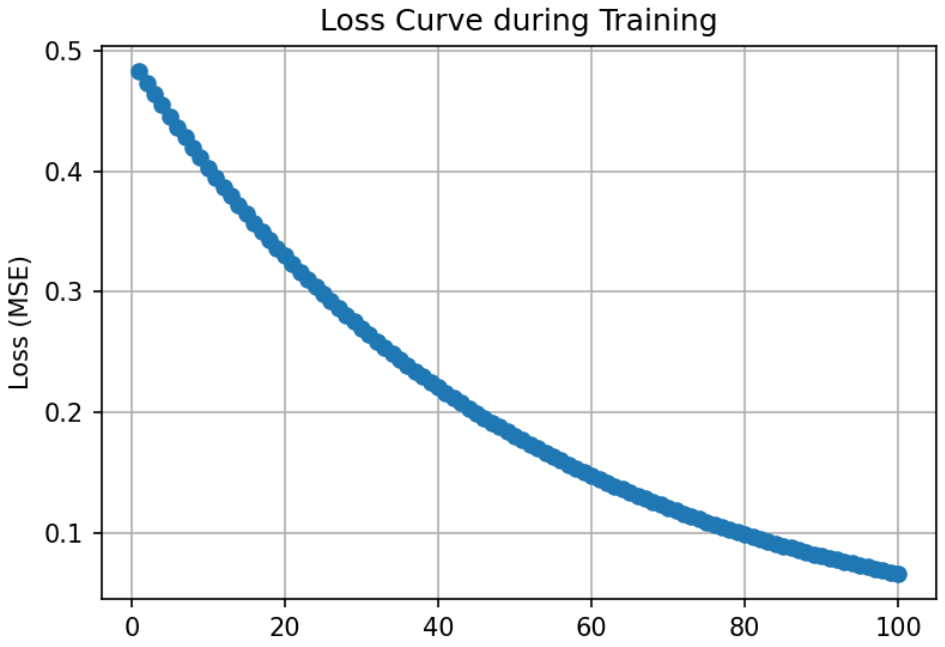
C03_linear_reg_ReLU_242.py

Compact code for Linear Regression (2-3-2)

```
# ---- 3) Training loop with loss tracking ----
losses = []
for step in range(1, 101):  # 100 steps for clearer trend
    optimizer.zero_grad()
    y = model(x)
    loss = criterion(y, t)
    loss.backward()
    optimizer.step()

    losses.append(loss.item())
    if step % 10 == 0:
        print(f"step {step:3d} | loss = {loss.item():.6f}")

# ---- 4) Plot loss curve ----
plt.figure(figsize=(6,4))
plt.plot(range(1, len(losses)+1), losses, marker='o')
plt.xlabel("Step")
plt.ylabel("Loss (MSE)")
plt.title("Loss Curve during Training")
plt.grid(True)
plt.show()
```

```
step  10 | loss = 0.403117
step  20 | loss = 0.329712
step  30 | loss = 0.269674
step  40 | loss = 0.220568
step  50 | loss = 0.180404
step  60 | loss = 0.147554
step  70 | loss = 0.120685
step  80 | loss = 0.098709
step  90 | loss = 0.080735
step 100 | loss = 0.066034
```





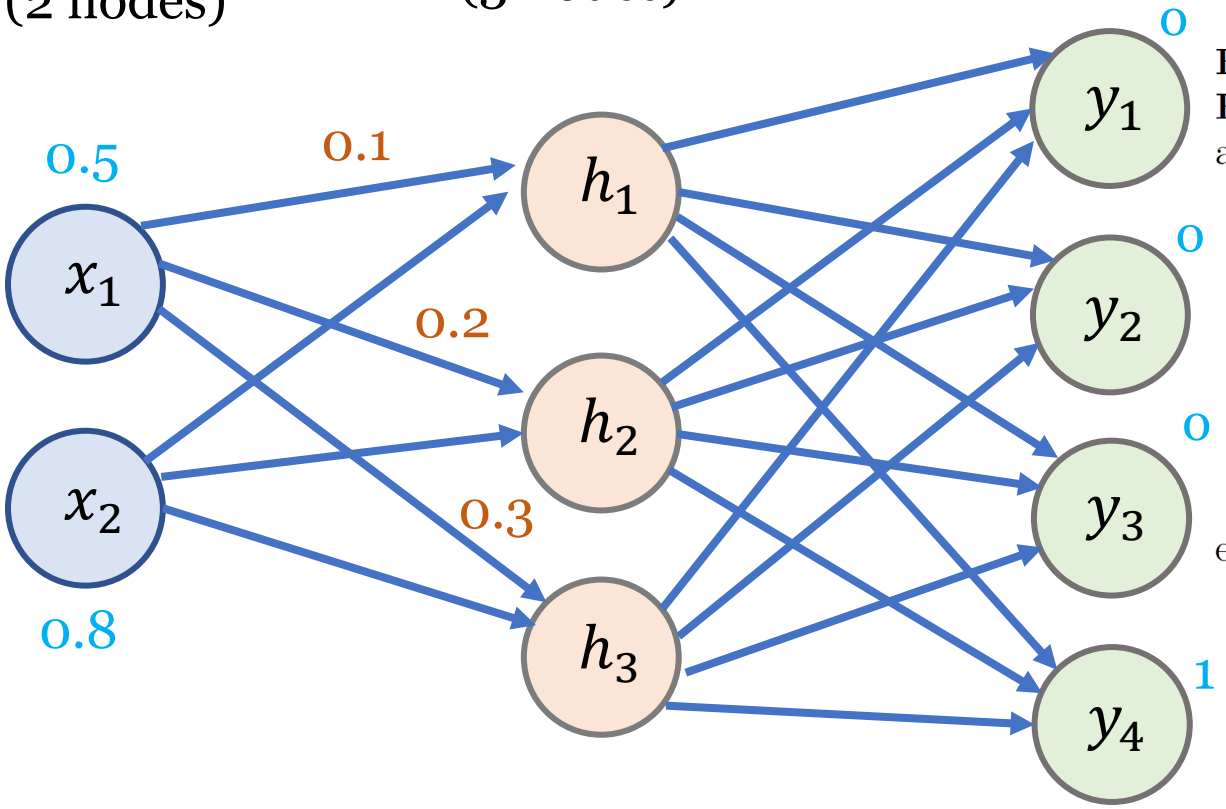
Backpropagation

Classification

Input layer
(2 nodes)

hidden layer
(3 nodes)

Ouput layer
(4 nodes)



Activation Functions: **ReLU**

Backpropagation computes gradients efficiently using the chain rule. We use **ReLU** in the hidden layer and **Softmax + Cross-Entropy** at the output for a 4-class classification task.

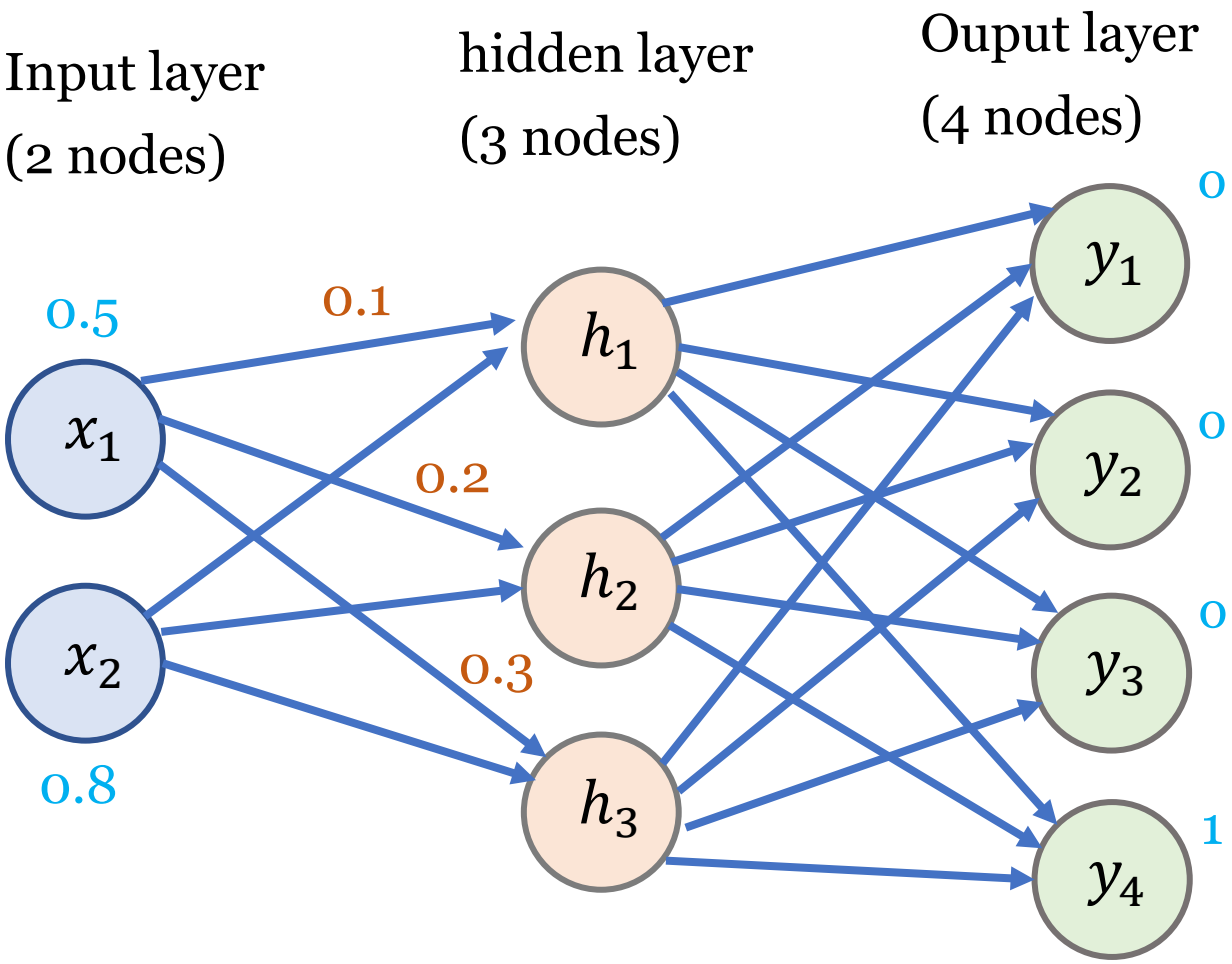
$$\mathrm{ReLU}(z) = \max(0, z), \qquad \mathrm{ReLU}'(z) = \begin{cases} 1, & z > 0, \\ 0, & z \le 0. \end{cases}$$

$$\mathrm{softmax}(z)_i = \frac{e^{z_i}}{\sum_j e^{z_j}}, \qquad E = -\sum_{i=1}^{4} t_i \log(o_i).$$

We keep the order *local gradient* $\times$ *upstream gradient* throughout, and denote error signals by q_o (output) and q_h (hidden).

Backpropagation

Classification



Activation Functions: **ReLU**

Initial Network State (explicit values)

- **Input (row vector):**
  $x = \begin{pmatrix} 0.5 & 0.8 \end{pmatrix}$
- **Target (one-hot, 4 classes):**
  $t = \begin{pmatrix} 0 & 0 & 0 & 1 \end{pmatrix}$
- **Weights:**

$$W_{ih} = \begin{pmatrix} 0.1 & 0.2 & 0.3 \\ 0.4 & 0.5 & 0.6 \end{pmatrix}, \quad W_{ho} = \begin{pmatrix} 0.7 & 0.8 & 0.5 & 0.2 \\ 0.9 & 0.1 & 0.4 & 0.3 \\ 0.2 & 0.3 & 0.6 & 0.7 \end{pmatrix}$$

- **Biases (row vectors):**
  $B_h = \begin{pmatrix} 0.2 & 0.4 & 0.6 \end{pmatrix}, \quad B_o = \begin{pmatrix} 0.5 & 0.7 & 0.2 & 0.1 \end{pmatrix}$

Backpropagation

Classification

Activation Functions: **ReLU**

1. Forward Pass (row-vector convention)

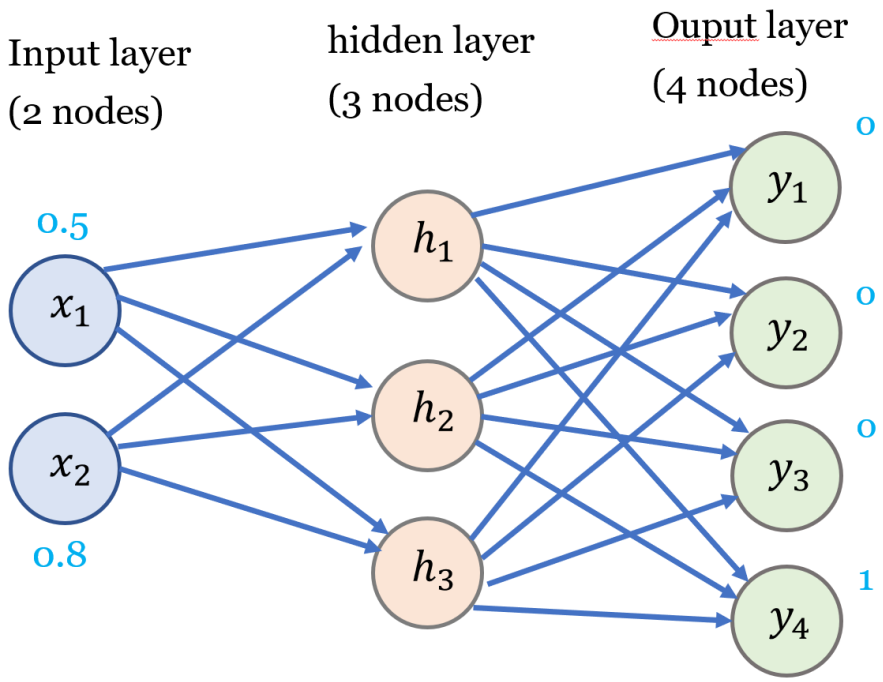
$$\text{net}_h = xW_{ih} + B_h = \begin{pmatrix} 0.57 & 0.90 & 1.23 \end{pmatrix},$$

$$\text{out}_h = \text{ReLU}(\text{net}_h) = \begin{pmatrix} 0.57 & 0.90 & 1.23 \end{pmatrix},$$

$$\text{net}_o = \text{out}_h W_{ho} + B_o = \begin{pmatrix} 1.955 & 1.615 & 1.583 & 1.345 \end{pmatrix},$$

$$\text{out}_o = \text{softmax}(\text{net}_o) \approx \begin{pmatrix} 0.33962 & 0.24173 & 0.23412 & 0.18453 \end{pmatrix},$$

$$E = -\sum_{i=1}^{4} t_i \log(o_i) = -\log(0.18453) \approx 1.68993.$$



Backpropagation

Classification

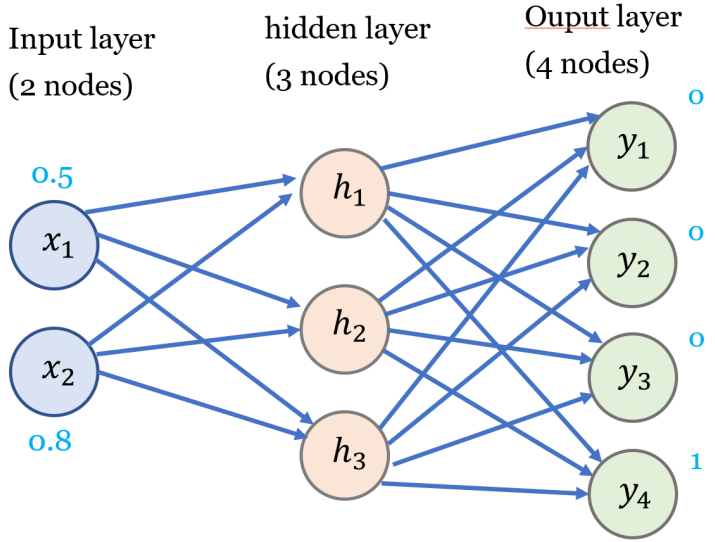
Activation Functions: **ReLU**

2. Backward Pass (local × upstream)

2.1. Output layer (define the output error signal). For Softmax + Cross-Entropy, the Jacobian–vector product simplifies to

$$q_o := \underbrace{\frac{\partial \,\mathrm{softmax}}{\partial \mathrm{net}_o}}_{\text{local}} \times \underbrace{\frac{\partial E}{\partial \mathrm{out}_o}}_{\text{upstream}} = \mathrm{out}_o - t$$

$$q_o \approx \begin{pmatrix} 0.33962 & 0.24173 & 0.23412 & -0.81547 \end{pmatrix}, \qquad \frac{\partial E}{\partial \mathrm{net}_o} = q_o$$



Deriving the Softmax Jacobian Entry $\frac{\partial \, \mathrm{out}_{o,i}}{\partial z_k} = \mathrm{out}_{o,i} \left(\delta_{ik} - \mathrm{out}_{o,k}\right)$

Let $z = \mathrm{net}_o \in \mathbb{R}^{1\times K}$ be the logits and

$$\mathrm{out}_{o,i} = \mathrm{softmax}(z)_i = \frac{e^{z_i}}{\sum_{j=1}^{K} e^{z_j}} \, .$$

Define the normalizer

$$S := \sum_{j=1}^{K} e^{z_j}.$$

Then

$$\mathrm{out}_{o,i} = \frac{e^{z_i}}{S}.$$

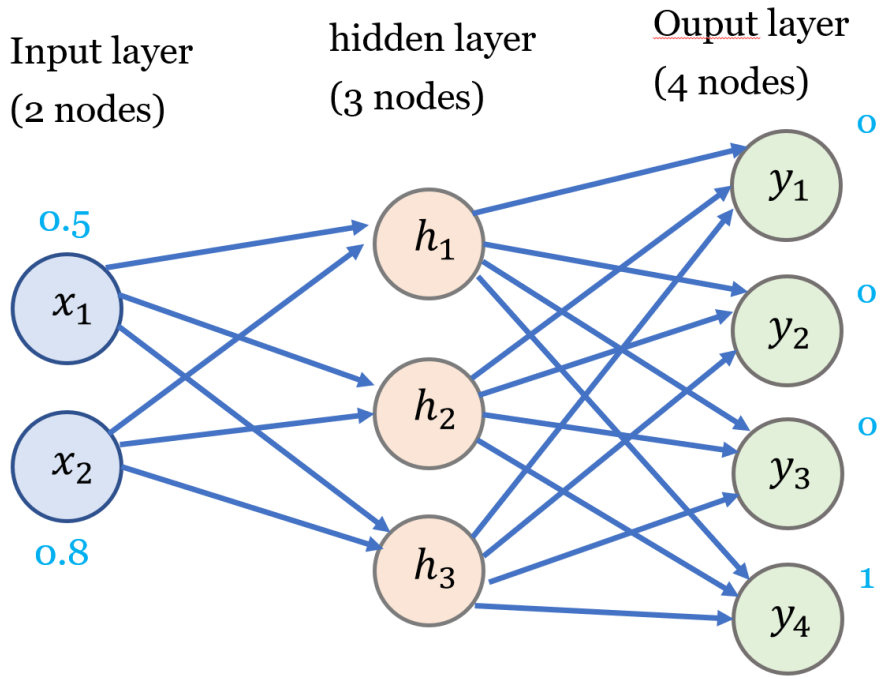
Step 1: Differentiate using the quotient rule. For fixed indices i, k,

$$\frac{\partial \, \mathrm{out}_{o,i}}{\partial z_k} = \frac{S \, \frac{\partial}{\partial z_k}\left(e^{z_i}\right) - e^{z_i} \, \frac{\partial S}{\partial z_k}}{S^2}.$$

Step 2: Compute the two partial derivatives.

$$\frac{\partial}{\partial z_k}\left(e^{z_i}\right) = e^{z_i} \, \delta_{ik} \quad \text{(since it is 0 unless } i = k\text{)},$$

$$\frac{\partial S}{\partial z_k} = \frac{\partial}{\partial z_k}\left(\sum_{j=1}^{K} e^{z_j}\right) = e^{z_k}.$$



Step 3: Substitute and simplify.

$$\frac{\partial \,\mathrm{out}_{o,i}}{\partial z_k} = \frac{S\,(e^{z_i}\delta_{ik}) - e^{z_i}\,e^{z_k}}{S^2} = \frac{e^{z_i}}{S^2}\Big(S\,\delta_{ik} - e^{z_k}\Big).$$

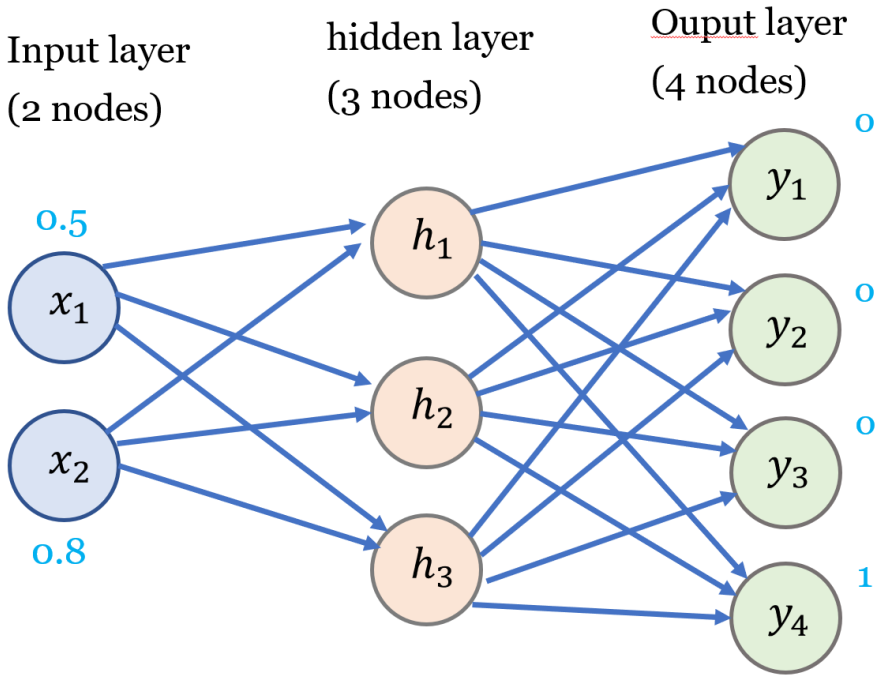
Recognize $\mathrm{out}_{o,i} = \dfrac{e^{z_i}}{S}$ and $\mathrm{out}_{o,k} = \dfrac{e^{z_k}}{S}$:

$$\frac{\partial \,\mathrm{out}_{o,i}}{\partial z_k} = \Big(\frac{e^{z_i}}{S}\Big)\Big(\delta_{ik} - \frac{e^{z_k}}{S}\Big) = \mathrm{out}_{o,i}\,(\delta_{ik} - \mathrm{out}_{o,k}).$$

Sanity check by cases.

$$k = i: \quad \frac{\partial \,\mathrm{out}_{o,i}}{\partial z_i} = \mathrm{out}_{o,i}\,(1 - \mathrm{out}_{o,i}),$$

$$k \neq i: \quad \frac{\partial \,\mathrm{out}_{o,i}}{\partial z_k} = -\,\mathrm{out}_{o,i}\,\mathrm{out}_{o,k}.$$



Compact matrix form. With $y = \mathrm{out}_o \in \mathbb{R}^{1\times K}$, the full Jacobian is

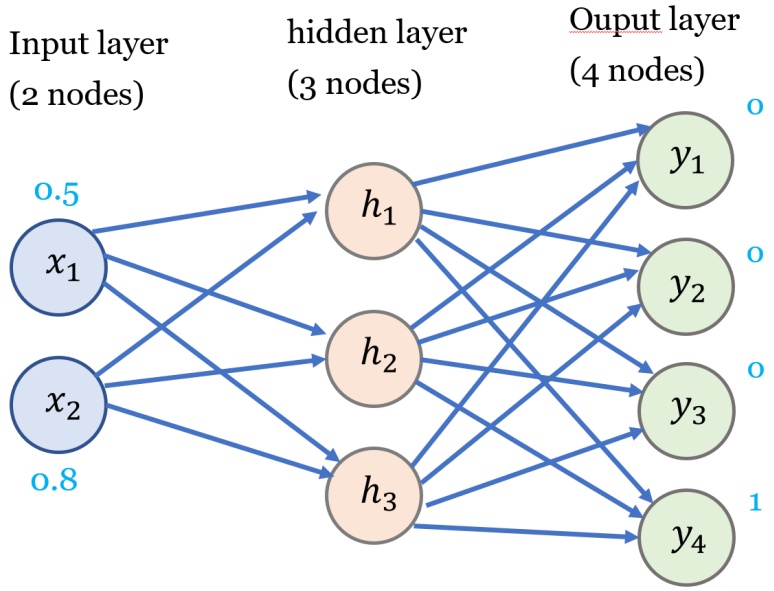
$$J_{\mathrm{sm}} = \frac{\partial\,\mathrm{softmax}(z)}{\partial z} = \mathrm{diag}(y) - y^{\top} y,$$

whose (i,k)-th entry is exactly $y_i(\delta_{ik} - y_k)$.

Backpropagation

```
import torch
import torch.nn as nn
import torch.optim as optim
import matplotlib.pyplot as plt

torch.set_printoptions(precision=6, sci_mode=False)

# ---- 1) Model ----
class MyModel(nn.Module):
    def __init__(self):
        super().__init__()
        self.fc1 = nn.Linear(2, 3, bias=True)  # input(2) -> hidden(3)
        self.fc2 = nn.Linear(3, 4, bias=True)  # hidden(3) -> output(4)  (logits)
        self.reset_to_given_values()

    def reset_to_given_values(self):
        W_ih = torch.tensor([[0.1, 0.2, 0.3],
                             [0.4, 0.5, 0.6]], dtype=torch.float32)  # (2x3)
        B_h  = torch.tensor([0.2, 0.4, 0.6], dtype=torch.float32)     # (3,)
        W_ho = torch.tensor([[0.7, 0.8, 0.5, 0.2],
                             [0.9, 0.1, 0.4, 0.3],
                             [0.2, 0.3, 0.6, 0.7]], dtype=torch.float32)  # (3x4)
        B_o  = torch.tensor([0.5, 0.7, 0.2, 0.1], dtype=torch.float32)    # (4,)

        with torch.no_grad():
            # nn.Linear stores weight as (out_features, in_features)
            self.fc1.weight.copy_(W_ih.t())  # (3,2)
            self.fc1.bias.copy_(B_h)
            self.fc2.weight.copy_(W_ho.t())  # (4,3)
            self.fc2.bias.copy_(B_o)

    def forward(self, x):      # x: (N,2)
        h = torch.relu(self.fc1(x))
        logits = self.fc2(h)   # raw scores for CrossEntropyLoss
        return logits
```

Classification

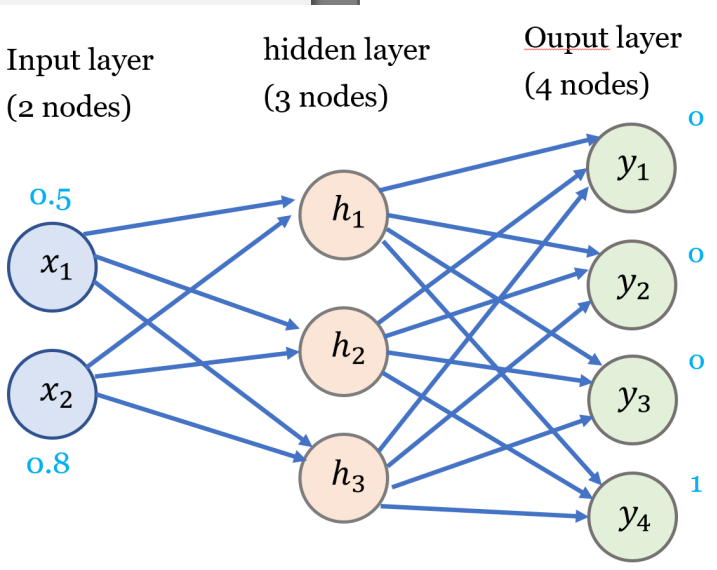


c03_backprop_classification_ReLU_242.py

Backpropagation

Classification

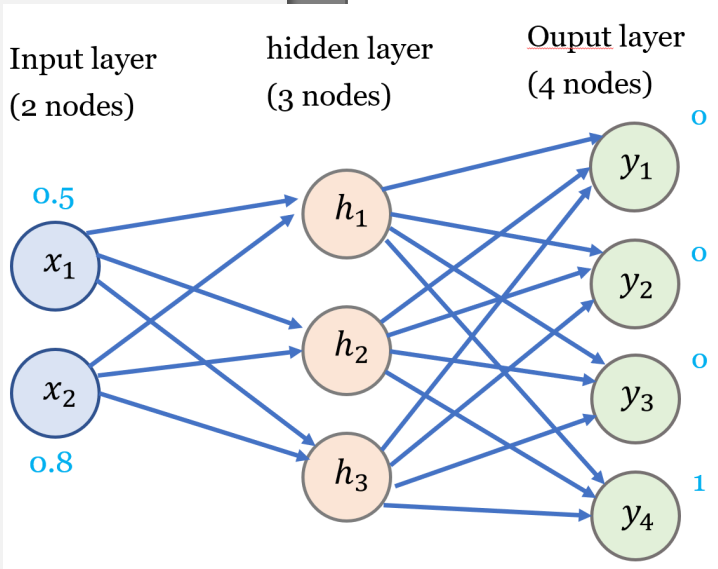
```
# ---- 2) Data / Loss / Optimizer ----
model = MyModel()

# Batch of one sample to play nicely with CrossEntropyLoss
x = torch.tensor([[0.5, 0.8]], dtype=torch.float32)  # (1,2)
target_index = torch.tensor([3], dtype=torch.long)   # (1,)

criterion = nn.CrossEntropyLoss(reduction='mean')  # applies log-softmax + NLL
learning_rate = 0.10
optimizer = optim.SGD(model.parameters(), lr=learning_rate)

# ---- 3) print parameters ----
def print_params(step, mdl):
    with torch.no_grad():
        Wih = mdl.fc1.weight.t()
        Bh  = mdl.fc1.bias
        Who = mdl.fc2.weight.t()
        Bo  = mdl.fc2.bias
        print(f"\n[After step {step}]")
        print("W_ih =\n", Wih.numpy())
        print("B_h  = ", Bh.numpy())
        print("W_ho =\n", Who.numpy())
        print("B_o  = ", Bo.numpy())

# Helper to view softmax probs
def softmax_probs(logits):
    return torch.softmax(logits, dim=1)  # softmax per row (across classes) --> rows sum to 1
```



Backpropagation

Classification

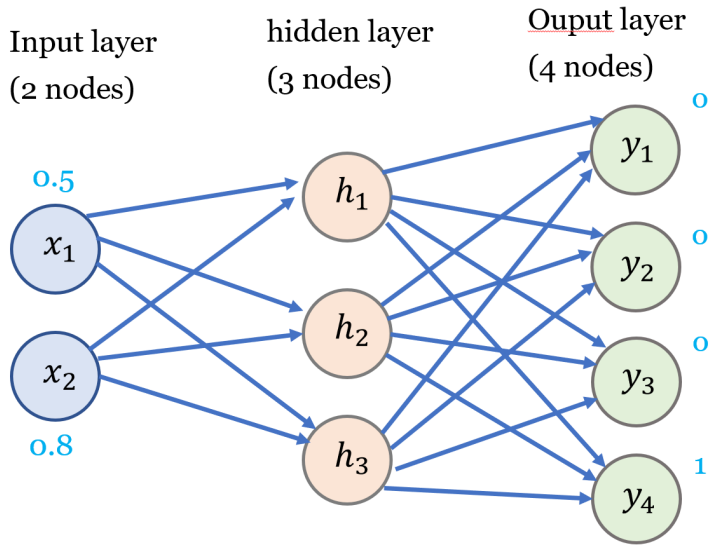
```
# ---- 4) Training loop with loss tracking ----
losses = []

for step in range(1, 21):  # 20 steps to see the curve
    optimizer.zero_grad()
    logits = model(x)                   # (1,4)
    probs  = softmax_probs(logits)      # for inspection only
    loss   = criterion(logits, target_index)  # CE on logits
    loss.backward()

    # (Optional) On step 1, verify q_o = probs - one_hot equals grad wrt output bias
    if step == 1:
        with torch.no_grad():
            one_hot = torch.zeros_like(probs)
            one_hot[0, target_index.item()] = 1.0
            q_o = probs - one_hot  # shape (1,4)
            print("\n[Step 1: gradient check at output layer]")
            print("logits =", logits.detach().squeeze(0))
            print("probs  =", probs.detach().squeeze(0))
            print("q_o (= probs - one_hot)        =", q_o.squeeze(0))
            print("grad wrt output bias (fc2.bias)=", model.fc2.bias.grad)

    optimizer.step()

    losses.append(loss.item())
    print(f"step {step} | loss = {loss.item():.6f} | probs = {probs.detach().numpy().round(6)}")

    if step % 5 == 0:
        print_params(step, model)
```



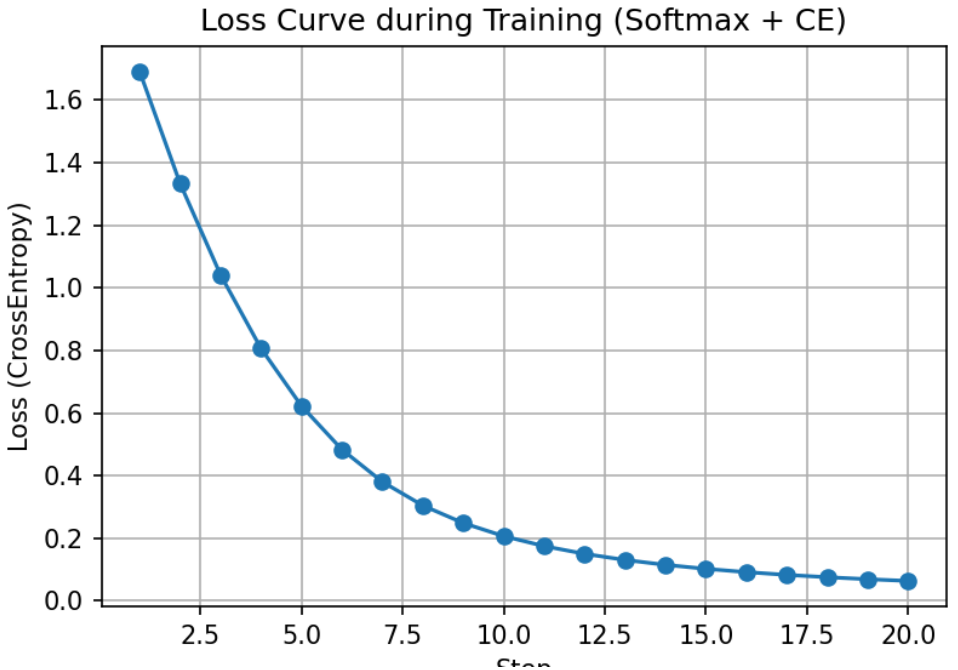
Backpropagation

Classification

```
# ---- 5) Plot loss curve ----
plt.figure(figsize=(6,4))
plt.plot(range(1, len(losses)+1), losses, marker='o')
plt.xlabel("Step")
plt.ylabel("Loss (CrossEntropy)")
plt.title("Loss Curve during Training (Softmax + CE)")
plt.grid(True)
plt.show()
```




```
[After step 5]
W_ih =
 [[-0.04336868  0.05882919  0.27575284]
 [ 0.17061011  0.2741267   0.5612046 ]]
B_h  =  [-0.08673736  0.11765838  0.5515057 ]
W_ho =
 [[ 0.571718    0.7558604 ]
 [ 0.6565286   0.02737943]
 [-0.19934106  0.19679347]]
B_o  =  [0.16177835 0.6148201 ]
step 6 | loss = 0.004628
step 7 | loss = 0.002013
step 8 | loss = 0.000912
step 9 | loss = 0.000425
step 10 | loss = 0.000202
```



Thank you!